第一章：绪论

嵌入式系统定义

嵌入式系统是嵌入到对象体系中的，用于执行独立功能 的专用计算机系统。

以应用为中心，以计算机，通信，控制等技术为基础，采用可剪裁软硬件，适用于对功能，可靠性，成本，体积，功耗等有严格要求的专用计算机系统。

嵌入式系统三要素

- **嵌入性** -> 满足环境要求
- **专用性** -> 满足配置要求
- **计算机系统** -> 满足控制要求

嵌入式系统与桌面通用系统的区别

- 运行的任务是**专用而确定**的
- 桌面通用系统需要支持**大量，需求多样的**应用程序
- 对**实时性**有较高的要求
- 对**可靠性**要求高
- 有**功耗约束**
- **内核小，可用资源少**
- 嵌入式系统的开发需要专用工具和特殊方法（**交叉编译，交叉链接**）

嵌入式系统通用结构

一般由 **嵌入式处理器，外围硬件设备，嵌入式操作系统（可选），以及用户的应用软件构成**

分类

按照字长分类：

- 4位系统
- 8位系统
- 16位系统
- 32位系统
- 64位系统

实时性需求分类：

- 非实时系统
- 软实时系统
- 硬实时系统

按系统的复杂程度分类：

- 小型系统
- 中性系统

- 复杂系统

第二章：ARM微处理器与编程模型

计算机体系结构

（1）冯诺依曼结构：将程序和数据存放在同一存储器的不同存储地址，顺序执行指令。
执行过程：取指令（or数据）-> 分析指令 -> 执行指令

（2）哈佛结构：将程序和数据分开存放在不同的存储器中，并行执行指令。
执行过程：取指令（and数据）-> 分析指令 -> 执行指令

计算机指令系统

CISC:复杂指令集计算机

RISC:精简指令集计算机

类别	CISC	RISC
指令系统	指令数量很多	较少，通常少于100
执行时间	有些指令执行时间很长，如整块的存储器内容拷贝；或将多个寄存器的内容拷贝到存贮器	没有较长执行时间的指令
编码长度	编码长度可变，1-15字节	编码长度固定，通常为4个字节
寻址方式	寻址方式多样	简单寻址
操作	可以对存储器和寄存器进行算术和逻辑操作	只能对寄存器进行算术和逻辑操作，Load/Store体系结构
编译	难以用优化编译器生成高效的目标代码程序	采用优化编译技术，生成高效的目标代码程序

流水线技术

使得几条指令可以并行执行。

提高CPU运行效率。

ARM含义

ARM三种含义

- 一个公司的名称
- 一类微处理器的通称
- 一种技术的名字

命名规则

- T：支持 Thumb 指令集
- D:片上调试
- M：支持长乘法
- I：提供片上断点和调试点
- E：增加了 DSP 指令
- J：Java 加速器
- S：可综合，提供 VHDL 或 Verilog 语言设计文件。

ARM微处理器结构

ARMCPU(内核)+外部设备（部件） == ARM芯片

ARM内核结构：

微处理器 == 运算部件 + 控制部件 + 寄存器组 + 总线

详细：包括 ALU、桶形移位器、乘法器、浮点部件（可选）、指令译码及控制逻辑、指令流水线、数据/地址寄存器、状态寄存器、总线等。

ARM微处理器运行模式

处理器模式	**用途**
USR- User mode（用户模式）	ARM处理器正常的程序执行状态
FIQ- Fast Interrupt mode（快速中断模式）	用于高速数据传输或通道处理
IRQ- interrupt mode （外部中断模式）	用户通用的中断处理
SVC- supervisor mode（管理模式）	操作系统使用的保护模式，处理软件中断
ABT- Abort mode（数据访问终止模式）	当数据或指令预取中止时进入该模式，可用于虚拟机存储器和存储器保护
UND-Undefined mode（未定义指令模式）	当未定义指令执行时进入该模式，可用户支持硬件协处理器的软件仿真
SYS-System mode（系统模式）	运行具有特权的操作系统任务

除了第一个**用户模式**，其他六个通称为**特权模式。**

用户模式与**特权模式**对比：

特权模式：程序可以访问所有的系统资源，可以任意地进行处理器模式切换

用户模式：大多数应用运行在这里，不能访问某些被保护的系统资源，不能**直接**切换到其他特权模式，但是可以通过异常**间接**实现模式切换

除了**用户模式**和**系统模式**，其他五个统称为**异常模式。**

对比：不能由异常直接进入，且使用完全相同的寄存器组。

系统模式：特权模式，没有用户模式的限制。OS在该模式下可以访问用户模式的寄存器；特权任务可以使用该模式访问一些受控的资源。

模式的切换方法：

运行模式可以直接或间接地通过程序切换进入。

当特定异常出新时，处理器进入对应的模式

处理器模式	备　注
USR（用户模式）	不能直接切换到其它模式
SYS（系统模式）	与用户模式类似，但具有可以直接切换到其它模式等特权
FIQ（快速中断模式）	FIQ异常响应时进入此模式
IRQ（外部中断模式）	IRQ异常响应时进入此模式
SVC（管理模式）	系统复位和软件中断响应时进入此模式
ABT（数据访问中止模式）	在ARM7TDMI中几乎没有用处
UND（未定义指令模式）	未定义指令异常响应时进入此模式

ARM工作状态

ARM状态：执行32位对齐的ARM指令

Thumb状态：执行16位的，半字对齐的Thumb指令

★**注意**：两种状态之间的切换，**不影响 处理器的运行模式**或寄**存器的内容。**

互相切换：

当寄存器Rm的状态位**bit[0]=1**时，可以通过执行 “BX Rm”指令,使微处理器从**ARM状态→Thumb状态。**

当寄存器Rm的状态位**bit[0]=0**时，可以通过执行 “BX Rm”指令,使微处理器从**Thumb状态→ARM状态。**

0状态：ARM

1状态：Thumb

ARM指令集和Thumb指令集

ARM指令集：

- 具有固定宽度32位指令，需要**4字节对齐**
- 在有限的存储空间中，存放指令少，代码密度低

Thumb指令集：

- Thumb 指令**2 字节对齐**

区别：

两种指令集的区别

	ARM指令集	Thumb指令集
协处理器指令	有	无
信号量指令	有	无
访问CPSR/SPSR指令	有	无
乘加/64位乘法指令	有	无
跳转指令	三种跳转指令都可以指定条件	**除B指令外，都是无条件跳转**；跳转范围小
数据处理指令	三个操作数	两个操作数；访问寄存器R8~R15受到一定限制
单寄存器加载/存储指令	无限制	只能访问寄存器R0~R7
批量寄存器加载/存储指令	无限制	只能访问R0~R7的子集

ARM寄存器组织

- ARM 状态下：37 个寄存器（**31 个通用寄存器+6 个状态寄存器**）
- Thumb 状态下：27 个寄存器（**21 个通用寄存器+6 个状态寄存器**）是Thumb状态的子集

31个通用寄存器：

- R0~R7：**未分组寄存器**，所有 7 种运行模式下，指向相同的物理寄存器，用于保存数据或者地址，**未被用作特殊的用途**

- R8~R12（ARM特有）：

（**Thumb 状态下不能使用这组寄存器，这就是 Thumb 为什么少了 10 个寄存器的原因**）

- R13：常用作**堆栈指针**，除用户模式与系统模式共用同一个物理寄存器，其他模式下都对应各自独立的物理寄存器，每种运行模式都有自己独立的物理寄存器R13
- R14：**子程序链接寄存器（链接寄存器 LR）。可以保持子程序或者异常处理后的返回地址。** 除用户模式与系统模式共用同一个物理寄存器，其他模式下都对应各自独立的物理寄存器
- R15：**程序计数器 PC。**所有 7 种运行模式下，都指向同一个物理寄存器。**pc的值为正在执行的命令的地址值+8，即正在执行的命令的后两条。-多级流水线技术。**

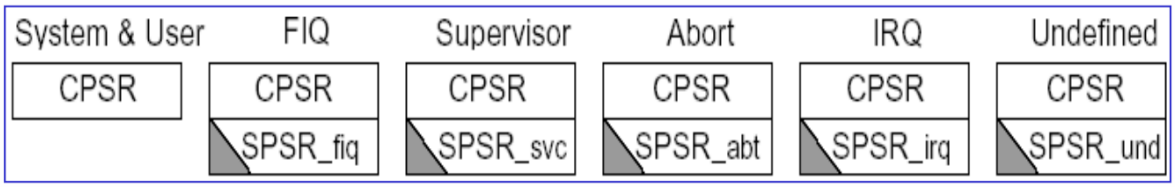
ARM和Thumb状态下：8个通用寄存器(R0 ~ R7)完全相同，CPSR和所有的SPSR完全相同，**只是Thumb状态下没有R8-R12。**

6个状态寄存器：

- **当前程序状态寄存器CPSR** (Current Program Status Register) ：**所有运行模式下都指向相同的物理寄存器。**可在任何运行模式下被访问。
- **备份程序状态寄存器SPSR** (Saved Program Status Register)：每种异常模式都有自己的专 用物理寄存器，异常发生时，保存CPSR；异常退出时用于恢复CPSR，**用户模式和系统模式下无效（访问时结果未知）**

状态寄存器（6个）

- 当前程序状态寄存器CPSR (Current Program Status Register)
  - ✓ 所有运行模式下都指向相同的物理寄存器。可在任何运行模式下被访问。



- 备份程序状态寄存器SPSR (Saved Program Status Register)
  - ✓ 每种异常模式都有自己的专用物理寄存器。
  - ✓ 异常发生时，保存CPSR；异常退出时用于恢复CPSR。
  - ✓ 用户模式和系统模式下无效（访问时结果未知）。

标志位：

标志位	含　义
N	**当用两个补码表示的带符号数进行运算时，N=1 表示运算的结果为负数；N=0 表示运算的结果为正数或零；**
Z	**Z=1 表示运算的结果为零；Z=0 表示运算的结果为非零；**
C	**加法运算结果最高位进位时，C=1，否则C=0； 减法运算最高位借位时，C=0，否则C=1； 移位操作指令，C为从最高位最后移出的位； 其它指令，C的值通常不改变。**
V	**加/减法运算指令，若符号位溢出，则V=1，否则V=0； 其它指令，V的值通常不改变。**
Q	**在ARM v5及以上版本的E系列处理器中，Q标志指示DSP运算指令是否溢出。在其他版本中，Q标志位无定义。**

N、Z、C、V标志位：它们的内容由算术或逻辑运算的结果决定，通常用于决定某条指令是否可以被执行。

- ARM状态：**绝大多数的指令都是有条件执行的。**
- Thumb状态：**仅有分支指令（B指令）是有条件执行的。**

ARM 微处理器存储器格式

大端格式：字数据的**高字节**存储在**低地址**中，而字数据的**低字节**存放在**高地址**中。

大端格式

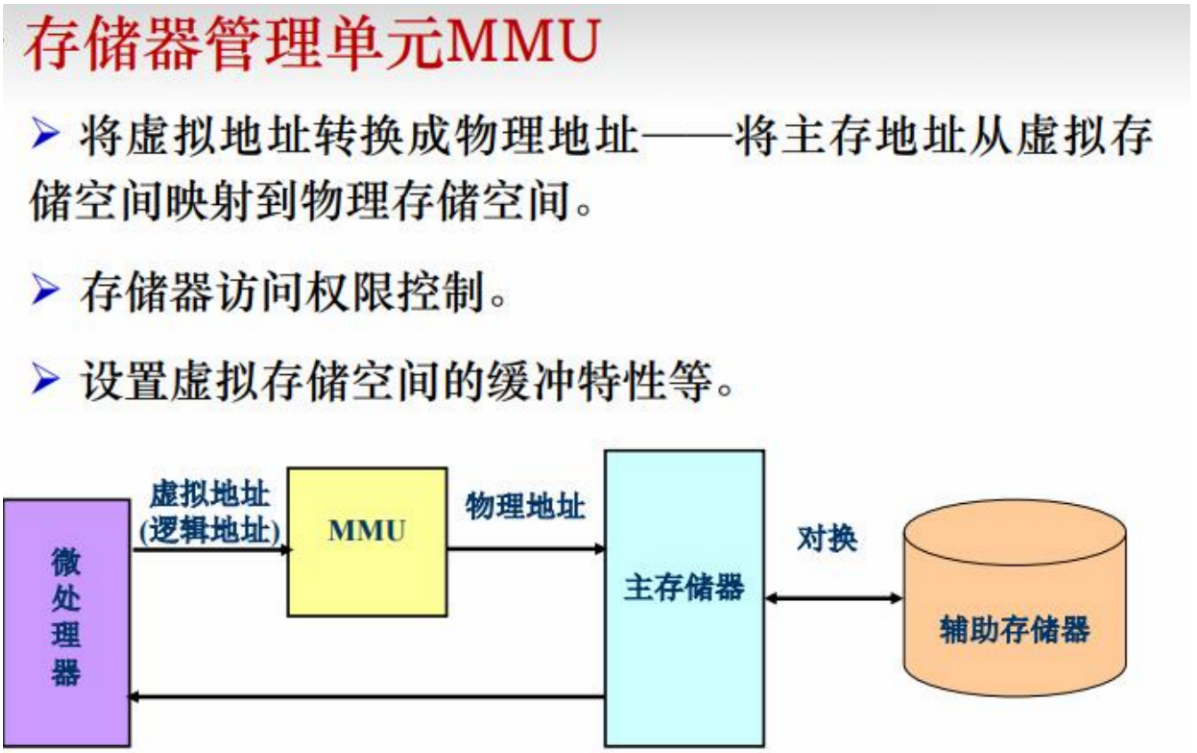
- 字数据的高字节存储在低地址中，而字数据的低字节则存放在高地址中。

小端格式：字数据的**高字节**存储在**高地址**中，而字数据的**低字节**存放在**低地址**中。

小端格式

- 字数据的高字节存储在高地址中，而字数据的低字节则存放在低地址中。

存储器管理单元MMU（Memory Management Unit）



虚拟存储管理

虚拟存储管理的三种方式：

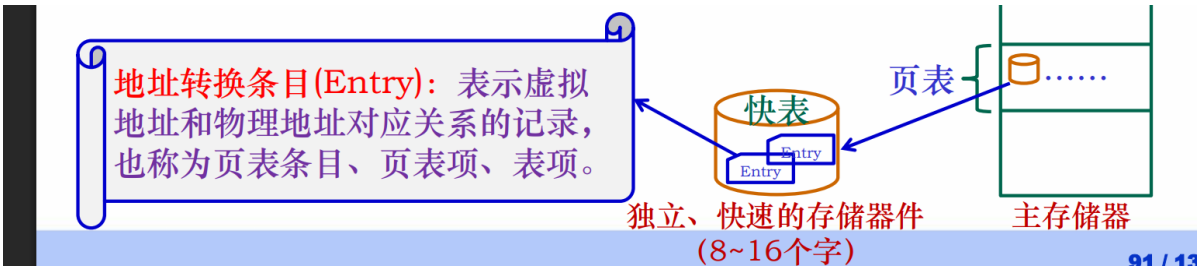
1. 分段式存储管理
2. 分页式存储管理 （只讲这个）
3. 段页式存储管理

分页式存储管理

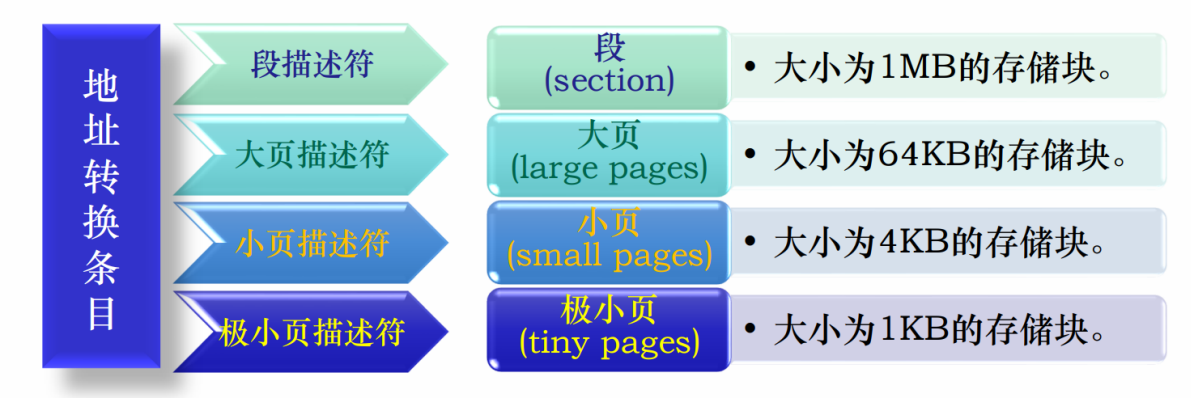
把**虚拟存储空间**分成一个个固定大小的*页(Page)*，把物**理主存储**的空间也分成同样大小的一个个**页框(Frame)。**

通过查询存放在主存中的**页表(Page Table)**，来实现**虚拟地址到物理地址的转换。**——对照表

由于查询页表代价很大，故采用**快表技术(Translation Lookaside Buffer, TLB)来提高地址转换速率**



存储块类型



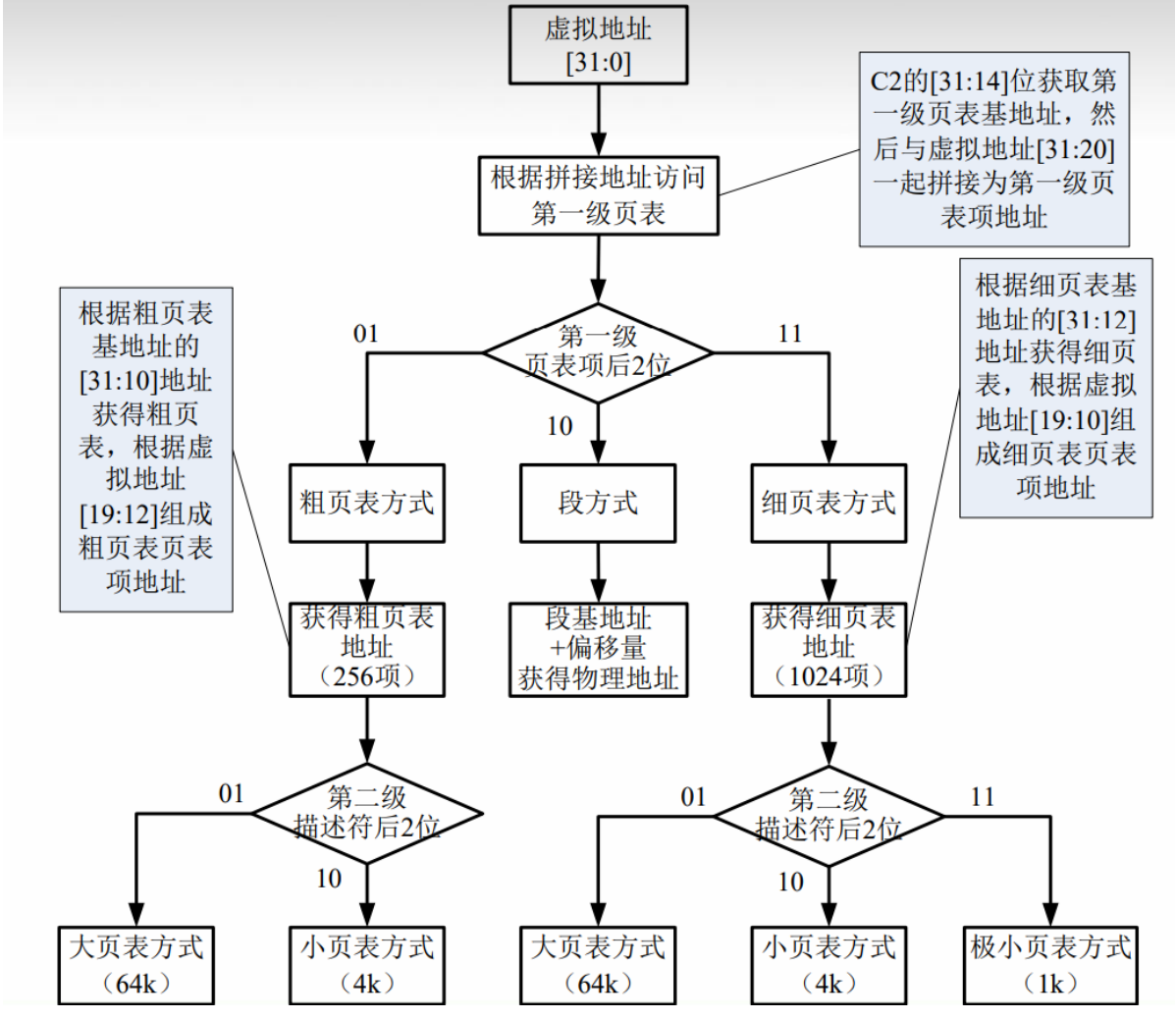
- 两级页表结构：**一级页表 L1** + **二级页表 L2**
- **一级页表L1**：4096 条地址转换条目（段），占用内存 16KB
- **二级页表 L2**：分别由**粗页表**和**细页表**构成
- **粗页表**：256 条地址转换条目（大页和小页），占用内存 1KB
- **细页表**：1024 条地址转换条目（大页、小页和极小页），占用内存 4KB

页表类型	页表项数	页表占用内存	支持页的单位
一级页表	4096	16k	1M
粗页表 (Coarse Page)	256	1k	4k或者64k
细页表 (Fine Page)	1024	4k	1k、4k或者64k

地址转换过程

地 址 转 换 过 程

- (1) 根据给定的虚拟地址查找一级页表，获得一个条目。
- (2) 若该条目是段描述符，则返回物理地址，转换结束。——单步搜索
- (3) 若该条目是粗页表目录项，则继续利用虚拟地址查找二级粗页表，获得大页描述符或小页描述符，返回物理地址，转换结束。——两步搜索
- (4) 若该条目是细页表目录项，则继续利用虚拟地址查找二级细页表，获得大页、小页或极小页描述符，返回物理地址，转换结束。——两步搜索
- (5) 其它情况出错。



异常处理

- 正常执行：每执行一条**ARM指令**，程序计数器寄存器 **(PC)的值加4个字节**；每执行一条**Thumb指令**，**PC的值加 2个字节。**整个过程按顺序执行。
- 通过跳转指令(B、BL、BLX和BX)，程序可以跳转到特 定的位置处(地址标号、子程序等）执行。
- 正常的程序执行流程发生暂时的停止，称之为异常。例 如：处理一个外部的中断请求

处理过程

当异常发生的时候，系统执行完当前命令后，**保存处理器当前状态（执行现场）**，然后跳转到异常处理程序执行。执行完成后，**程序恢复到所保存的执行现场，继续执行**，允许多个异常同时发生，按照固定优先级进行处理

关于异常的几个概念

异常向量地址(Exception Vectors)：异常出现后，强制从**异常类型对应的固定地址开始执行程序。**

异常向量表：由七个异常向量及其异常处理函数跳转关系组成的表。每个异常向量占4个字节。（★表现为一条跳转指令）

异常的类型

异常类型	优先级	
复位	1（最高优先级）	优先级降低
数据中止	2	
FIQ	3	
IRQ	4	
预取指令中止	5	
未定义指令	6	
SWI	7（最低优先级）	

异常类型	模式	低向量地址 (默认使用)	高向量地址
复位	管理	0x00000000	0xFFFF0000
未定义指令	未定义	0x00000004	0xFFFF0004
软件中断（SWI）	管理	0x00000008	0xFFFF0008
预取中止 （取指令存储器中止）	中止	0x0000000C	0xFFFF000C
数据中止 （数据访问存储器中止）	中止	0x00000010	0xFFFF0010
IRQ（中断）	IRQ	0x00000018	0xFFFF0018
FIQ（快速中断）	FIQ	0x0000001C	0xFFFF001C

异常类型	进入模式	具体含义	异常向量地址
复位	管理	复位电平有效时，产生复位异常，程序跳转到复位处理程序处执行。	0x00000000
未定义指令	未定义	遇到不能处理的指令时，产生未定义指令异常。	0x00000004
软件中断 (SWI)	管理	执行SWI指令产生，用于用户模式下的程序调用特权操作指令。	0x00000008
指令预取中止	中止	处理器预取指令的地址不存在，或该地址不允许当前指令访问，产生指令预取中止异常。	0x0000000C
数据访问中止	中止	处理器数据访问指令的地址不存在，或该地址不允许当前指令访问时，产生数据中止异常。	0x00000010
IRQ （中断）	IRQ	外部中断请求有效，且CPSR中的I位为0时产生IRQ异常。	0x00000018
FIQ （快速中断）	FIQ	快速中断请求有效，且CPSR中的F位为0时产生FIQ异常。	0x0000001C

具体步骤

分为**处理异常**和**异常返回**两部分

处理异常:

1. 将下一条地址存入响应的链接寄存器LR，以便程序在处理异常返回时能从正确的位置重新开始执行
2. 将CPSR复制到对应的SPSR
3. 根据异常类型，强制设置CPSR的运行模式位
4. 强制PC从相关的异常向量地址取下一条指令执行，从而跳转到相应的异常处理程序处，同时设置中断禁止位。

异常返回:

1. 将连接寄存器LR的值减去对应偏移量后送到PC中

2. 将SPSR复制回CPSR中

3. 若再进入异常处理时设置了中断禁止位，**要再次清除**

4. 可以认为应用程序总是从复位异常处理程序开始执行的，因此复位异常处理程序不需要返回

第三章：ARM指令集与汇编程序设计

ARM寻址方式

由**操作码**和**操作数**两部分组成

- 操作码：指示指令要执行的**具体操作**。用助记符（一般为 英文字母缩写）表示——指令助记符。
- 操作数：指出指令执行过程中的**操作对象**。可以用**符号**或 **符号地址**表示。

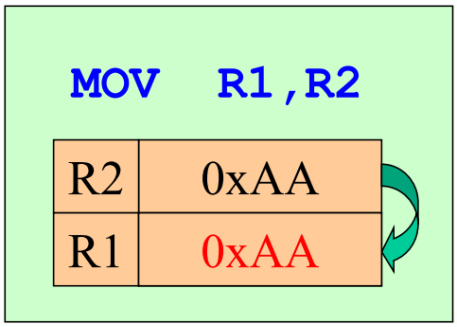
一共有八种寻址方式

（1）寄存器寻址

指令中操作数给出的是**寄存器编号**，指令执行时直接 取出寄存器的值来操作。

➢举例：

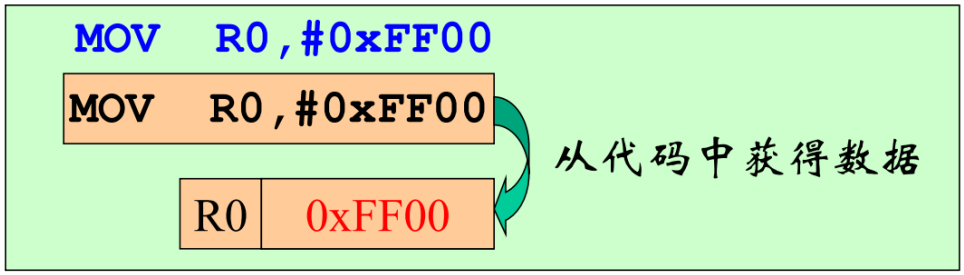
```
MOV  R1,R2          ;将R2的值存入R1，即R2→R1
SUB  R0,R1,R2       ; R0=R1-R2
```



（2）立即寻址

指令中操作数给出的是**寄存器编号**，指令执行时**直接 取出寄存器的值**来操作（称为**立即数**）。

```
SUBS  R0,R0,#1        ;R0=R0-1，且影响标志位
MOV   R0,#0xFF00      ;将立即数0xFF00装入R0寄存器
```



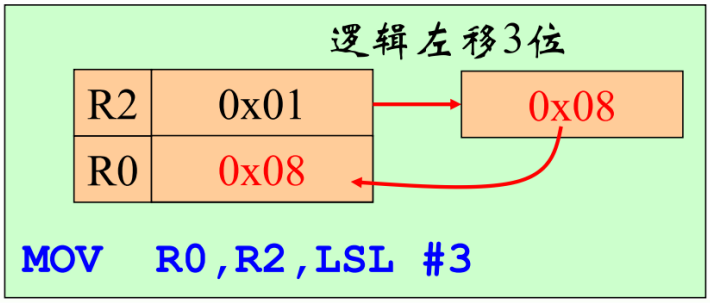
（3）寄存器移位寻址

指令中操作数在使用前，首先**执行移位操作。**（ARM特有方法）

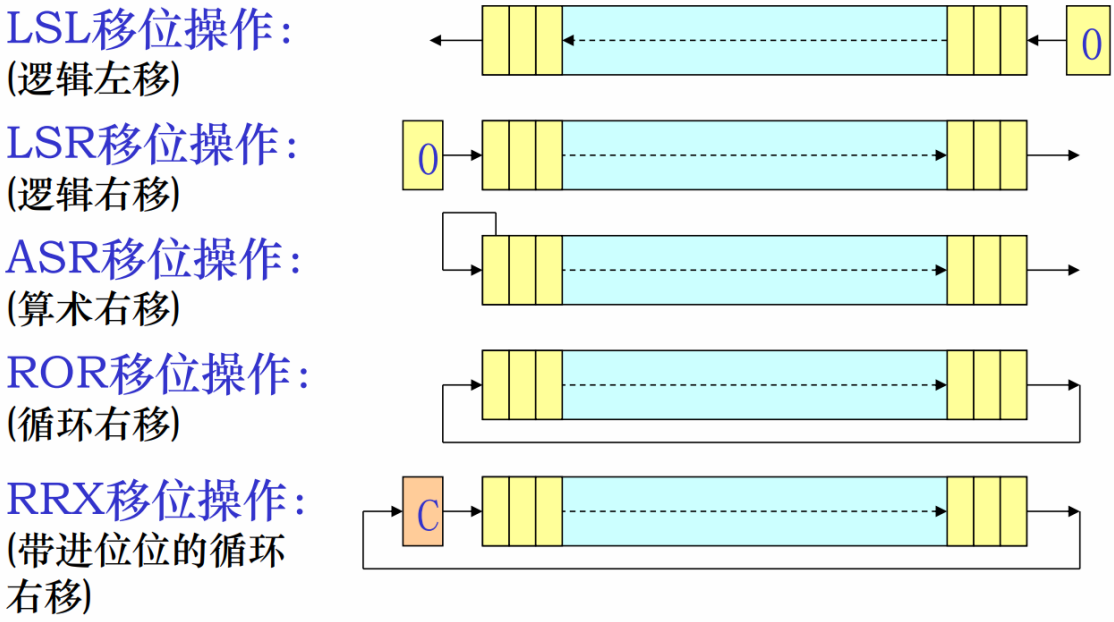
寄存器移位寻址，其实就是“寄存器寻址”的升级版——**只是多了一步“移位”处理**

➢**举例：** **（ARM指令集特有的寻址方式。）**

```
MOV  R0,R2,LSL #3      ;R2的值左移3位，结果放入R0，
                       ;即R0=R2×8  (2^3)
ANDS R1,R1,R2,LSL R3   ;R2的值左移R3位，然后和R1
                       ;相“与”操作，结果放入R1
```



ARM支持的移位操作



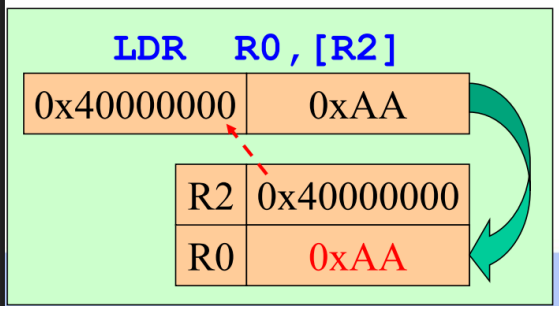
最后那个RRX移位操作：**RRX** = Rotate Right with eXtend 把进位标志 “塞” 进来，最低位“弹”出去（存入标志位C）

（4）寄存器间接寻址

指令操作数给出的是**通用寄存器的编号**，真正的数据 保存在**寄存器指定地址的存储单元**中，即**寄存器为操作数的地址指针。**

➢举例：

```
LDR   R0,[R2]        ;将R2指向的存储单元中的内容读出，放入R0
SWP   R1,R1,[R2]     ;将寄存器R1的值和R2指向的存储单元的内容交换
```



(5) 基址（或变址）寻址

将基址寄存器的内容与指令中给出的**偏移量相加**，形成操作数的**有效地址**。

➢举例：

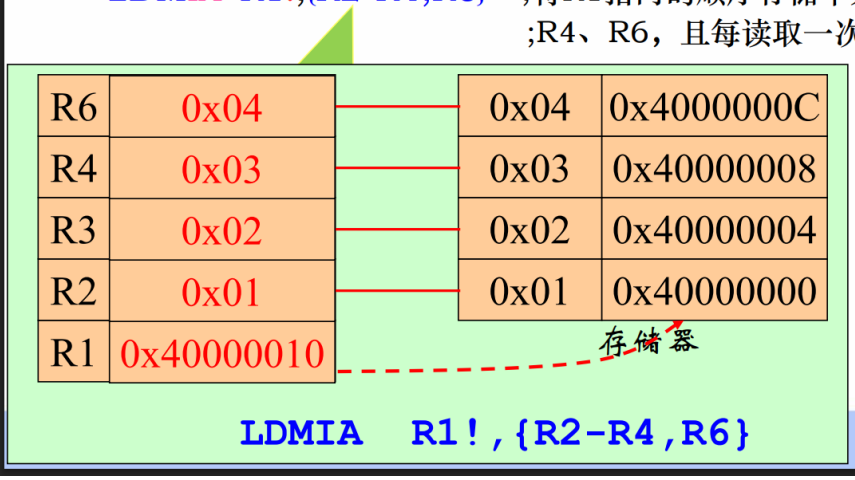
```
LDR   R2,[R3,#0x0C]  ;读取R3+0x0C指向的存储单元的内容，放入R2
STR   R1,[R0,#-4]!   ;把R1的值保存到R0=R0-4指向的存储单元
LDR   R0,[R3],#0x0C  ; R0= [R3] , R3=R3+0x0C
LDR   R0,[R1,R2]
```

(6) 多寄存器寻址

一次可传送几个寄存器值。允许一条指令传送不超过 **16** 个寄存器。

➢举例：

```
LDMIA  R1!,{R2-R4,R6}   ;将R1指向的顺序存储单元中的数据读出到R2～
                        ;R4、R6，且每读取一次，R1自动加4。
```



➢**举例：**

```
LDMIA  R1!,{R2-R4,R6}   ;将R1指向的顺序存储单元中的数据读出到R2～
                        ;R4、R6，且每读取一次，R1自动加4。
STMIA  R0!,{R2-R7,R12}  ;将寄存器R2～R7、R12的值保存到R0指向的
                        ; 顺序存储单元，且每保存一次，R0自动加4。
```

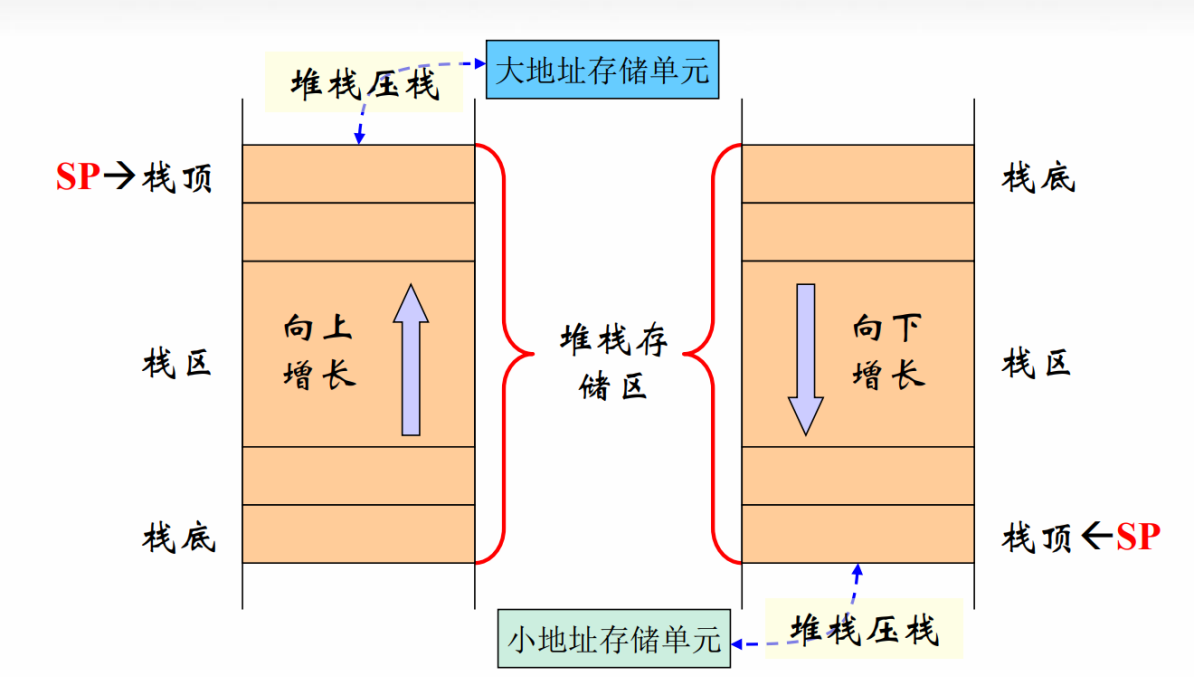
(7) 堆栈寻址

堆栈是存储器中一个按特定顺序(**先进后出** or **后进先出**)进行存取的区域。

隐含的寻址：使用专门的寄存器**(即堆栈指针SP)**指向一块存储区域。SP所指向的存储单元称为**栈顶。**

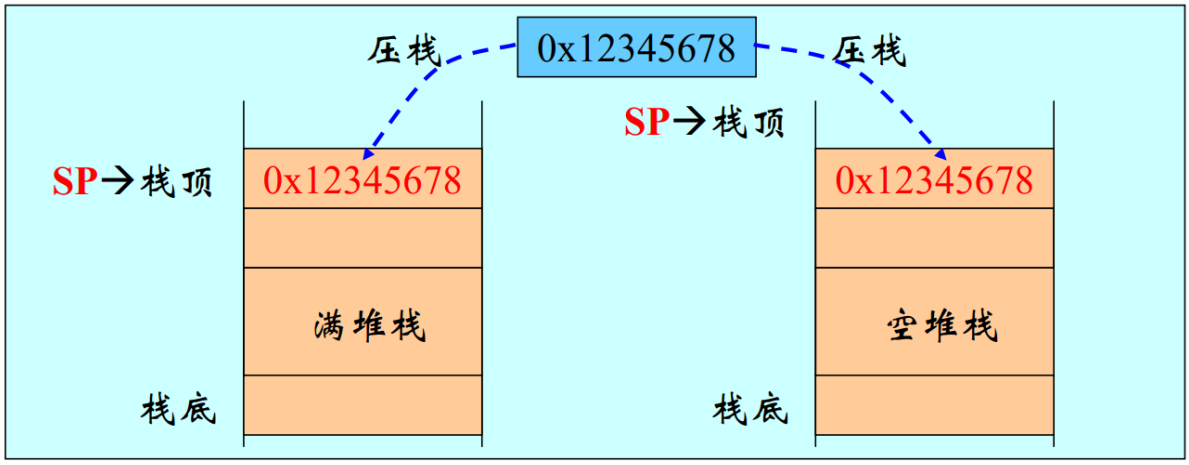
分类1（——根据 **入栈* 时SP的变化**）：

- **向上生长**：向高地址方向生长，称为**递增堆栈**
- **向下生长**：向低地址方向生长，称为**递减堆栈**



分类2（——根据**SP指向的内容**）：

- **满堆栈**：SP指向最后压入栈的**有效数据项**
- **空堆栈**：SP指向下一个待压入数据的**空位置**



➢四种类型的堆栈（两种分类组合）：

✓满递增（FA）：堆栈向上增长，堆栈指针指向存放有效数据项的最高地址。指令如LDMFA、STMFA等；

✓空递增（EA）：堆栈向上增长，堆栈指针指向堆栈上的第一个空位置。指令如LDMEA、STMEA等；

✓满递减（FD）：堆栈向下增长，堆栈指针指向存放有效数据项的最低地址。指令如LDMFD、STMFD等；

✓空递减（ED）：堆栈向下增长，堆栈指针向堆栈下的第一个空位置。指令如LDMED、STMED等。

LDM/STM八种模式

模式	说明	模式	说明
IA	每次传送后地址加4	FD	满递减堆栈
IB	每次传送前地址加4	ED	空递减堆栈
DA	每次传送后地址减4	FA	满递增堆栈
DB	每次传送前地址减4	EA	空递增堆栈
数据块传送操作（多寄存器寻址）		堆栈操作（堆栈寻址）	

(8) 相对寻址

是**基址寻址**的一种变通。由**程序计数器PC**提供**基准地址**，指令中的**操作数作为偏移量**，**两者相加得到的地址即为有效地址EA。**

ARM指令集

- RISC -> **加载/存储（Load/Store）型**
  - 指令集仅能处理寄存器中内容，而且结果要返回寄存器
  - 对存储器的访问需要通过专门的加载/存储指令来完成
- 带**T变种**的微处理器（如ARM7TDMI-S等）的指令集， 包括：
  - ARM指令集
  - Thumb指令集

ARM指令集与Thumb指令集的关系：

ARM指令集支持ARM核所有的特性，高效，快速的特点。**Thumb指令**集有灵活，小巧的特点。

基本格式

```
<opcode> {<cond>} {S} <Rd>, <Rn> {, <operand2>}
```

部分	含义	示例
`<opcode>`	**操作码（指令类型）**	`ADD`, `SUB`, `MOV`, `CMP`, `AND`, `ORR`, `EOR`, `RSB`, `ADC` 等
`{<cond>}`	**条件码**（可选）	`EQ`（等于），`NE`（不等于），`GT`（大于），`AL`（总是）等
`{S}`	**标志影响位S**（可选）	写 `S` 表示会更新 CPSR 的标志位（N/Z/C/V）
`<Rd>`	**目标寄存器**	指令执行后结果存入的寄存器
`<Rn>`	**第一个操作数寄存器**	常为源寄存器（比如加法的左边）
`<operand2>`	**第二操作数**（立即数、寄存器或带移位）	如 `#4`、`R1`、`R3, LSL #2` 等

条件码cond：

- 可以实现高效的逻辑操作，提高代码效率
- 所有的ARM指令都可以条件执行；而Thumb指令只有**B（跳转）指令**具有条件执行功能
- 缺省是**无条件（AL）**执行

条件码cond表：

操作码	条件助记符	标志	含义
0000	EQ	Z=1	相等
0001	NE	Z=0	不相等
0010	CS/HS	C=1	无符号数大于或等于
0011	CC/LO	C=0	无符号数小于
0100	MI	N=1	负数
0101	PL	N=0	正数或零
0110	VS	V=1	溢出
0111	VC	V=0	没有溢出
1000	HI	C=1,Z=0	无符号数大于
1001	LS	C=0,Z=1	无符号数小于或等于
1010	GE	N=V	有符号数大于或等于
1011	LT	N!=V	有符号数小于
1100	GT	Z=0,N=V	有符号数大于
1101	LE	Z=1,N!=V	有符号数小于或等于
1110	AL	任何	无条件执行（指令默认条件）
1111	NV	任何	从不执行(不要使用)

标志影响位S

默认情况下，**数据处理指令**不影响CPSR的条件标志位 (N、Z、C、V)，**但可以选择通过添加“S”来影响**

特殊：**比较指令**不需要添加“S”默认改变条件标志位。

指令集

- 存储器访问指令
- 分支指令（跳转指令）
- 数据处理指令
- 乘法指令
- 杂项指令
- 伪指令

1.存储器访问指令

➢用于对内存变量的访问、内存缓冲区数据的访问、查表、外围部件的控制操作等。

➢使用单寄存器加载指令加载数据到PC寄存器，可实现程序的跳转功能。

助记符	说明	操作	条件码位置
LDR Rd,addressing	加载（LoaD）字数据	Rd<-[addressing]	LDR{cond}
STR Rd,addressing	存储（STore）字数据	[addressing] <- Rd	STR{cond}
LDR**B** Rd,adderssing	加载**无符号字节**数据	Rd<-[addressing]	LDR{cond}**B**

同理，**H**就是**半字**， **SB**就是**有符号字节数据**

无论是LDR指令还是STR指令都可以用这类后缀

还有**LDM（LoaD Multiple）**和**STM（STore Multiple）**加载多个和存储多个

主要用途是现场保护、数据复制、参数传递等。

SWP指令：用于将一个内存单元(**地址在寄存器Rn中**)的内 容读取到一个**寄存器Rd**中，同时将另一个**寄存器Rm**的内容写入到该内存单元中。可用于实现信号量操作。

2.分支指令（跳转指令）

在ARM程序中有两种方式可以实现程序的跳转：

- 使用专门的跳转指令 - **±32MB地址空间跳转**
- 直接向程序计数器PC写入跳转的地址 - 可以在4GB地址空间的跳转

分支(跳转)指令包括以下4条指令：

助记符	说明	操作	条件码位置
B label	跳转指令	PC←label	B{cond}
BL label	带返回的跳转指令	LR←PC-4，PC←label	BL{cond}
BX Rm	带状态切换的跳转指令	PC←Rm，切换处理器状态	BX{cond}
BLX Rm	带返回和状态切换的跳转指令	LR←PC-4，PC←Rm，切换处理器状态	BLX{cond}

3.数据处理指令

数据处理指令 { 数据传送指令 / 算术/逻辑运算指令 / 比较指令

- 只能对寄存器进行操作，不能对存储器进行操作。
- 可选择使用S后缀，并影响状态标志。比较指令即使不使用S后缀，也会影响状态标志。

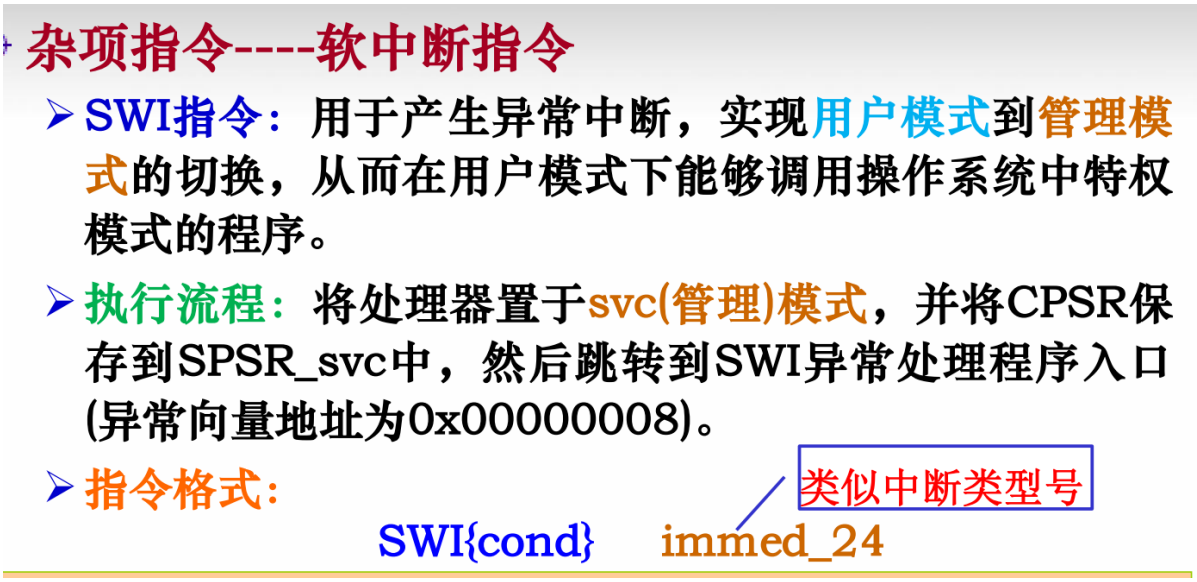
4.乘法指令

乘法指令

助记符	说明	操作	条件码位置
MUL Rd,Rm,Rs	32位乘法指令	Rd←Rm*Rs (Rd≠Rm)	MUL{cond}{S}
MLA Rd,Rm,Rs,Rn	32位乘加指令	Rd←Rm*Rs + Rn (Rd≠Rm)	MLA{cond}{S}
UMULL RdLo,RdHi,Rm,Rs	64位无符号乘法指令	(RdLo,RdHi)←Rm*Rs	UMULL{cond}{S}
UMLAL RdLo,RdHi,Rm,Rs	64位无符号乘加指令	(RdLo,RdHi)←Rm*Rs + (RdLo,RdHi)	UMLAL{cond}{S}
SMULL RdLo,RdHi,Rm,Rs	64位有符号乘法指令	(RdLo,RdHi)←Rm*Rs	SMULL{cond}{S}
SMLAL RdLo,RdHi,Rm,Rs	64位有符号乘加指令	(RdLo,RdHi)←Rm*Rs + (RdLo,RdHi)	SMLAL{cond}{S}

5.杂项指令

助记符	说明	操作	条件码位置
MRS Rd, psr	读状态寄存器指令	Rd←psr，psr为CPSR或SPSR	MRS{cond}
MSR psr_fields, Rd / #immed_8r	写状态寄存器指令	psr_fields←Rd/#immed_8r ， psr为CPSR或SPSR	MSR{cond}
SWI immed_24	软中断指令	产生软中断，处理器进入管理模式	SWI{cond}



6.伪指令

ADR/ADRL伪指令格式 ——小/中等范围的地址读取

```
ADR/ADRL{cond} register, expr
指令执行地址码   加载的目标寄存器   地址表达式
```

将基于PC相对偏移的**地址值**或基于寄存器相对偏移的**地址值**读取到目标寄存器中。

LDR伪指令格式 ——加载**32位立即数**或**地址值**到指定寄存器

```
LDR {cond} register, =[expr | label_expr]
指令执行地址码   加载的目标寄存器   基于PC的地址表达式或外部表达式
```

EQU伪指令格式 ——将一个**数值**或**寄存器名**赋给指定的符号名

其中：

- name：expr定义的符号名称
- expr：基于寄存器的地址值、程序中的标号、32位的 地址常量或者32位的**常量**
- type：当expr为32位常量时，可以使用type指示expr 数据的类型，取值为CODE32、CODE16和DATA。

NOP伪指令格式 ——空操作

会被替换成一些无意义的

ARM汇编程序设计

都是PPT的例子，没有新的知识点

Thumb指令集

- 是ARM指令压缩形式的子集。为了减少代码量提出，具有16位代码密度。
- 指令体系不完整，只支持通用功能。
- 必要时仍需要ARM指令（进入异常时）

与ARM指令的主要区别

- **只有B指令可以条件执行**，其他都不能条件执行
- 分支指令的跳转范围有更多限制
- 单寄存器访问指令（LDR/STR）只能操作R0-R7
- 多寄存器访问指令（LDM/STM）可对R0-R7的任何子集进行操作
- PUSH和POP指令（满递减）：R0-R7，PC

第四章：嵌入式存储器系统

嵌入式存储器分类

按用途分类：

主存：用来存放正在执行或者经常使用的程序代码和数据

- **存取速度快**
- **容量有限**
- 位于嵌入式微处理器芯片的内部或外部

辅助存储器：

- **存取速度慢**
- **容量大，成本低**
- 位于嵌入式微处理器芯片的外部

按存储介质分类：

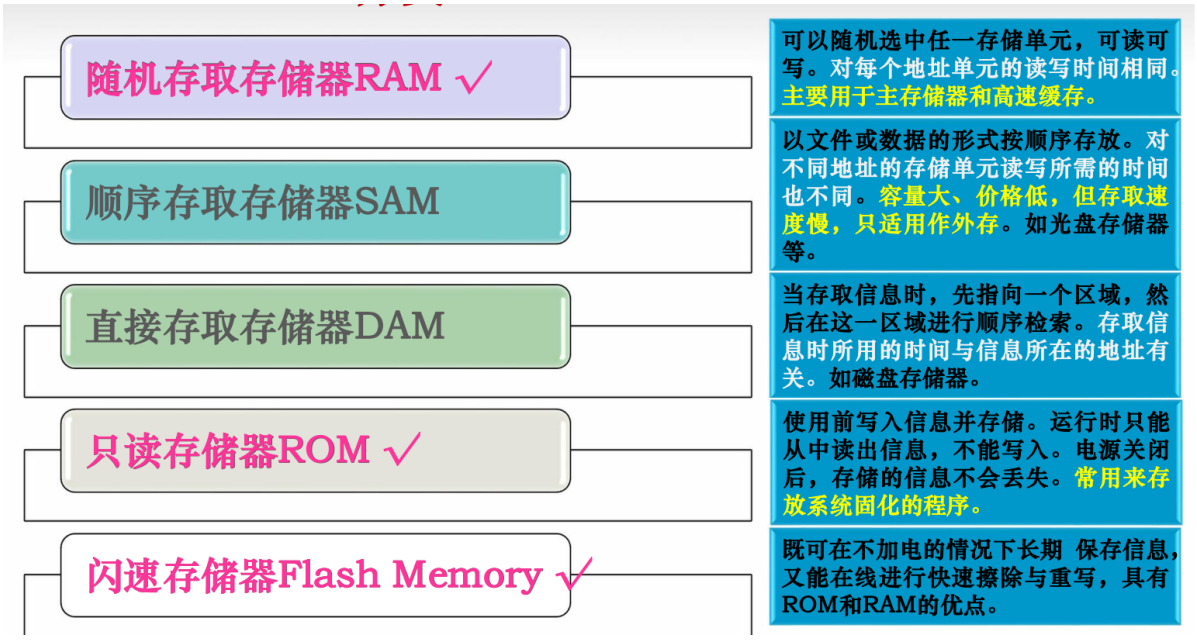
（1）半导体存储器 --- 主流

（2）光盘存储器

（3）磁表面存储器

后两者主要作为辅存

按存取方式分类：



然后RAM又可以细分，就跟计组的知识点差不多：

按存取方式分类（续）

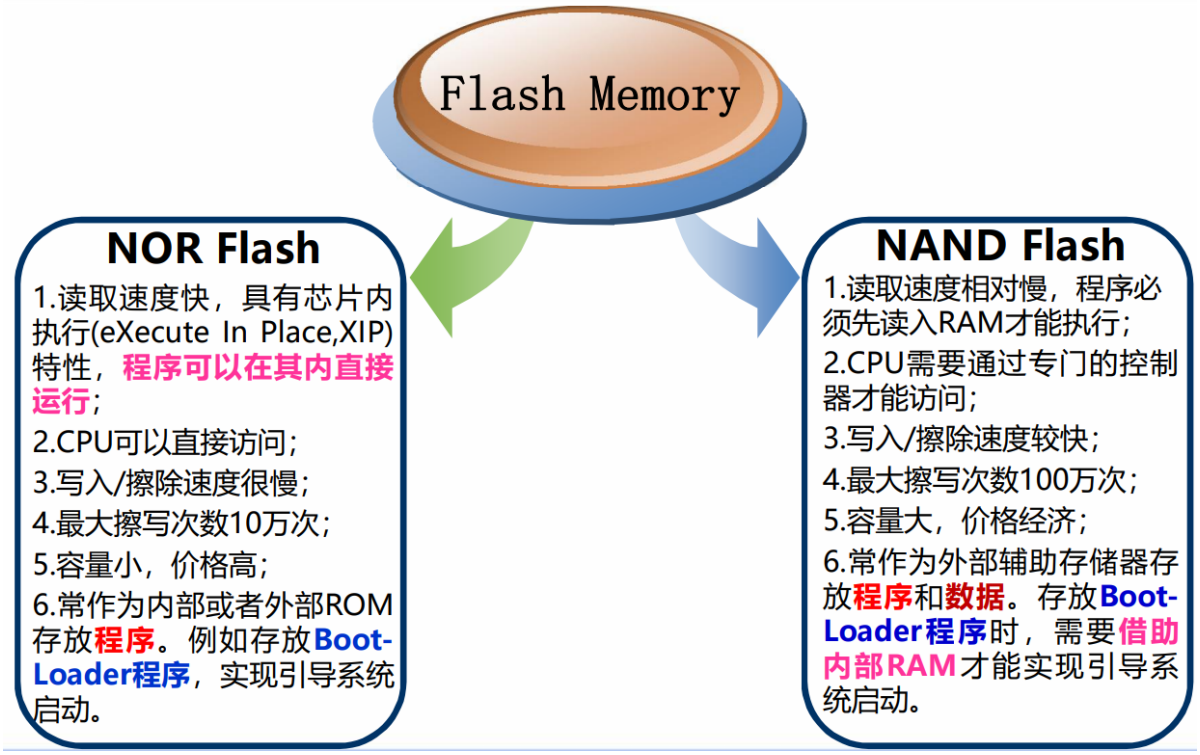
- 随机存取存储器RAM
  - 静态随机存取存储器SRAM（Static RAM）
    - ✓ 一个bit存储单元由多个晶体管耦合（双稳态触发器）而成。
    - ✓ 存储内容可以长时间保持，除非掉电或执行写操作。
    - ✓ 速度非常快，但集成度低，结构复杂，功耗大。
  - 动态随机存取存储器DRAM（Dynamic RAM）
    - ✓ 一个bit存储单元由一个场效应管及其栅极对地接一个电容组成。
    - ✓ 存储内容一定时间后会自动消失，需要周期性刷新——（充电）。
    - ✓ 速度比SRAM慢一些，但集成度高，成本低，功耗低。
  - 同步动态随机存取存储器SDRAM（Synchronous DRAM）
    - ✓ 将CPU和RAM共享一个时钟周期，两者以相同的速度同步工作，在每一个时钟脉冲的上升沿开始传输数据。
  - 双倍数据速率动态随机存取存储器DDR SDRAM（Double Data Rate SDRAM）
    - ✓ 允许在时钟脉冲的上升沿和下降沿两次传输数据，从而在不提高时钟频率的情况下能加倍提高SDRAM的传输速度。

ROM也是：

按存取方式分类（续）

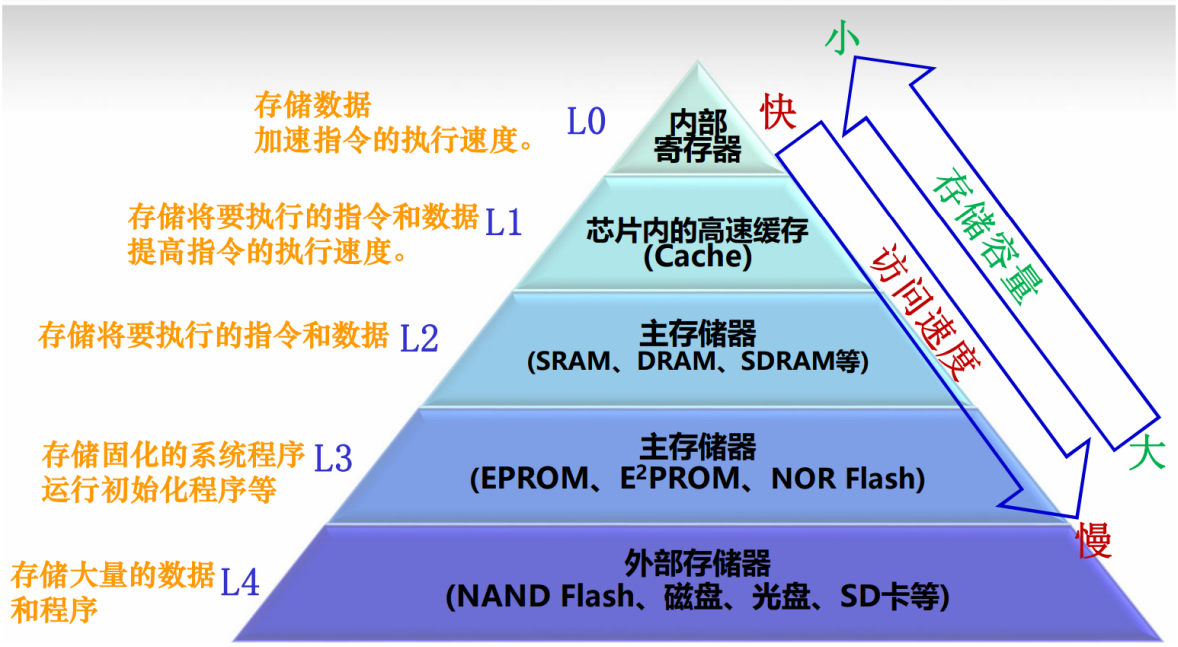
- 只读存储器ROM
  - 掩膜只读存储器MROM（Mask ROM）
    - ✓ 芯片制造时，在一个bit存储单元通过加入一个MOS管表示逻辑“0”。
    - ✓ 程序/数据在出厂前由制造商写入，之后不能修改。
    - ✓ 成本低，适用于定型产品的大批量生产。
  - 可编程只读存储器PROM（Programmable ROM）
    - ✓ 一个bit存储单元由二极管或者熔丝构成。
    - ✓ 使用专用编程设备，完成程序/数据的写入。但写入一次后就不能再修改——（一次可编程）。
  - 可擦除可编程只读存储器EPROM（Erasable PROM）
    - ✓ 使用专用紫外线设备，对已写入的内容进行多次擦除，再使用专用编程设备，完成程序/数据多次写入。
  - 电可擦除可编程只读存储器E^2PROM（Electrically Erasable PROM）
    - ✓ 施加高于工作电压的编程电压(12-24V)，再使用专用编程设备，完成程序/数据多次写入。
    - ✓ 可以在工作系统中在线进行。

还有最后一个 **Flash Memory** 非易失性存储器，可读可写



NOR Flash读取快，NAND Flash写入快

存储器系统结构



支持小端/大端模式。——通过编程设定

可寻址外部存储空间为1GB。

27根地址线ADDR0-ADDR26

8条片选线nGCS0-nGCS7

- 被分成8个Bank(Bank0～Bank7)，每个Bank为128MB。(8×128MB = 1024MB = 1GB)
  - ✓ Bank0的数据位宽只能是16或者32位(由引脚电平决定)，其它Bank可以编程设定为8、16或者32位。
  - ✓ Bank0～Bank5可以外接ROM、SRAM类型的存储器；Bank6和Bank7可以外接ROM、SRAM、SDRAM类型的存储器。
  - ✓ Bank6和Bank7的大小可以编程设定。
  - ✓ Bank0～Bank6有固定的起始地址；Bank7起始地址 = Bank6的末地址 + 1。
  - ✓ 所有Bank的访问周期可编程设定。

外部存储器芯片连接

本质上是存储器和微处理器之间**数据总线**， **地址总线**， **控制总线**。之间的连接。

数据总线和控制总线就按照对应的引脚连接即可。

下面主要讲地址线的连接：

地址总线的连接

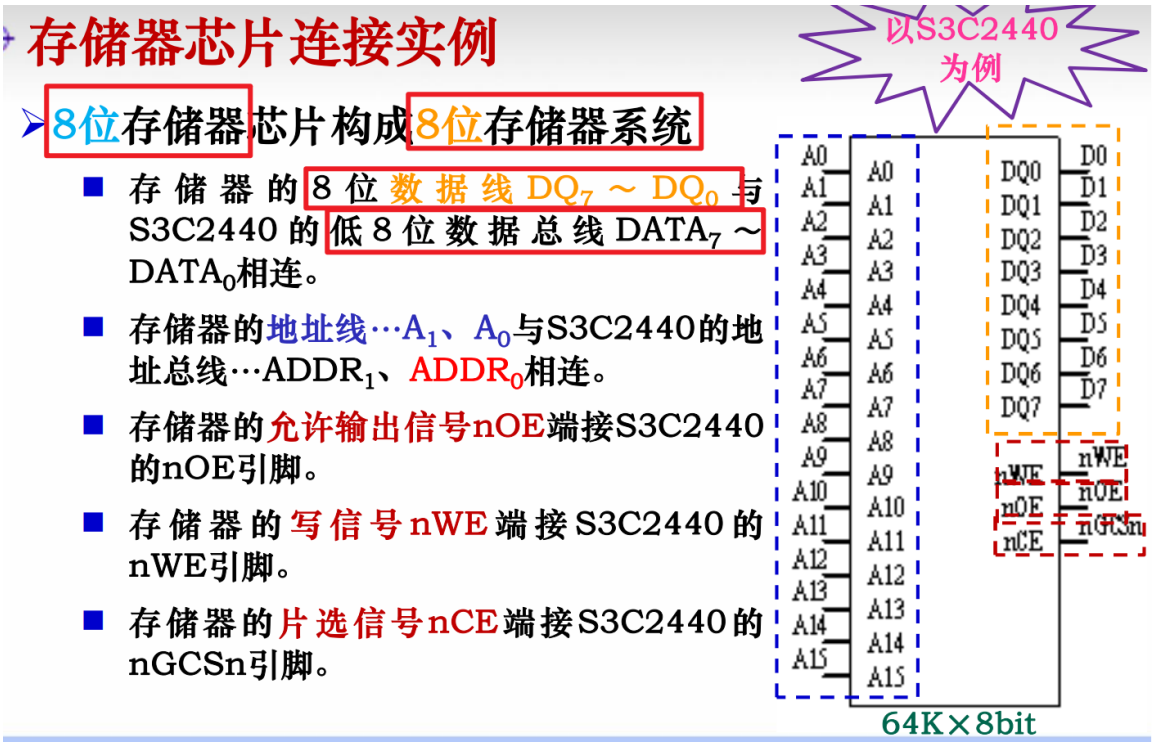
以S3C2440为例

➢与(存储)系统数据总线的位数有关。

存储器芯片的地址引脚	8 位存储系统 S3C2440的地址引脚	16 位存储系统 S3C2440的地址引脚	32 位存储系统 S3C2440的地址引脚
A_0	A_0	A_1	A_2
A_1	A_1	A_2	A_3
……	…	…	…
	8位存储器芯片	8/16位存储器芯片	8/16/32位存储器芯片

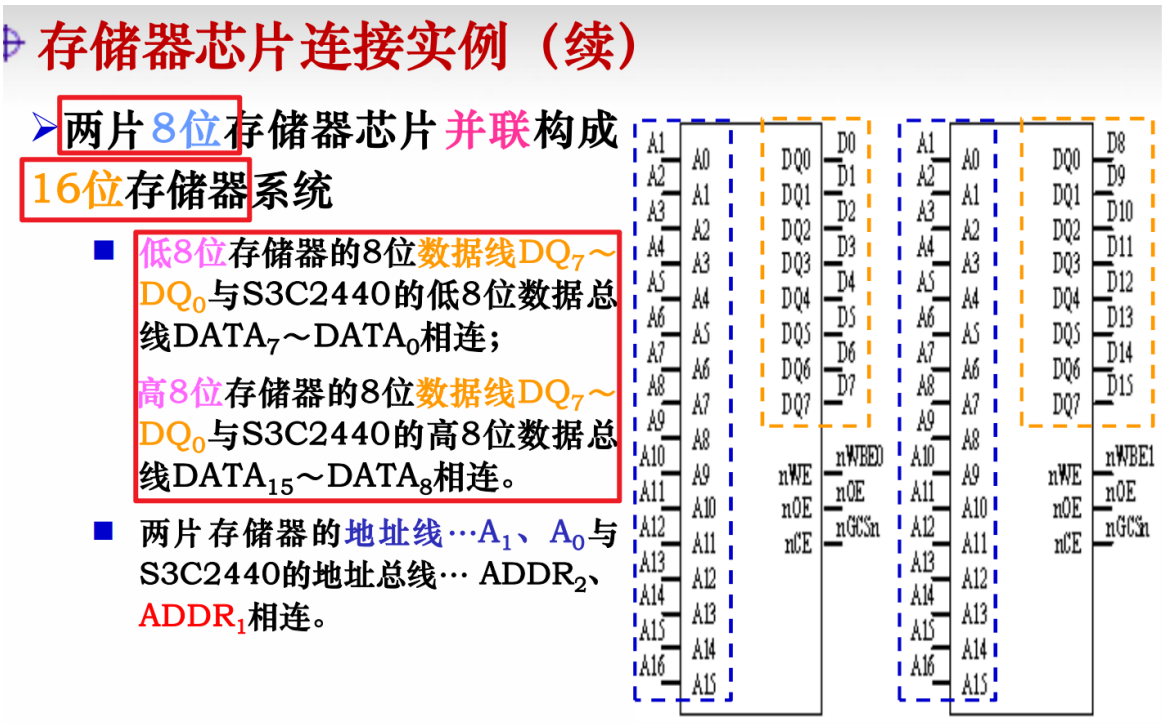
连接案例：

8位连接8位芯片：



没有位拓展，那就0-7 相对应即可

2片8位连接16位芯片（位拓展）：



注意看数据线的连接：

- 低八位的 7-0 连接芯片 7-0
- 高八位的 7-0 连接芯片15-8

存储器芯片连接实例（续）

➢两片8位存储器芯片并联构成16位存储器系统（续）

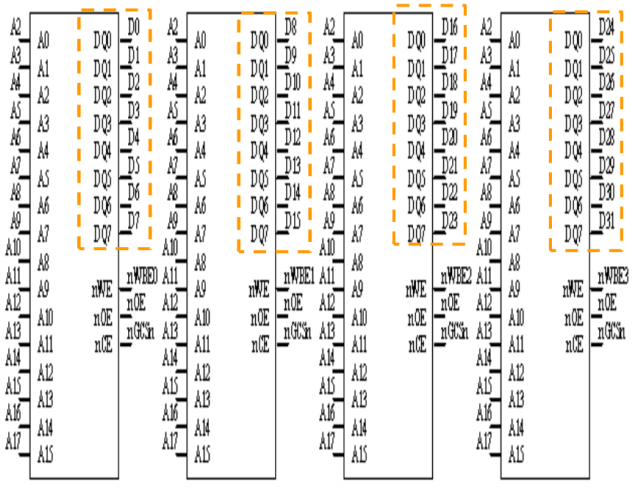
- 两片存储器的允许输出信号nOE端都接S3C2440的nOE引脚。
- 低8位存储器的写信号nWE端接S3C2440的$nWBE_0$引脚，用于写入低字节数据；

  高8位存储器的写信号nWE端接S3C2440的$nWBE_1$引脚，用于写入高字节数据。
- 两片存储器的片选信号nCE端都接S3C2440的nGCSn引脚。(作为一个整体配置到同一Bank中)

4片8位连接32位芯片：

存储器芯片连接实例（续）

➢四片8位存储器芯片并联构成32位存储器系统

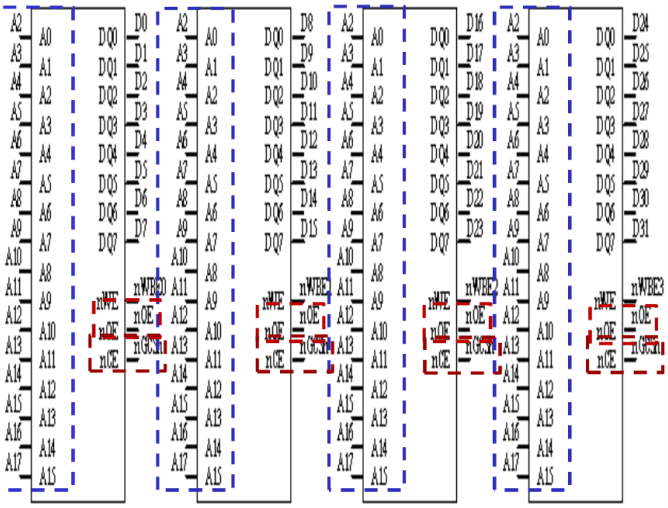


- 低8位存储器的8位数据线DQ_7～DQ_0与S3C2440的低8位数据总线$DATA_7$～$DATA_0$相连；
- 次低8位存储器的8位数据线DQ_7～DQ_0与S3C2440的次低8位数据总线$DATA_{15}$～$DATA_8$相连；
- 次高8位存储器的8位数据线DQ_7～DQ_0与S3C2440的次高8位数据总线$DATA_{23}$～$DATA_{16}$相连；
- 高8位存储器的8位数据线DQ_7～DQ_0与S3C2440的高8位数据总线$DATA_{31}$～$DATA_{24}$相连。

- 低八位的 7-0 连接芯片 7-0
- 次低八位的 7-0 连接芯片 15-8
- 次高八位的 7-0 连接芯片23-16
- 高八位的 7-0 连接芯片 31-24

存储器芯片连接实例（续）

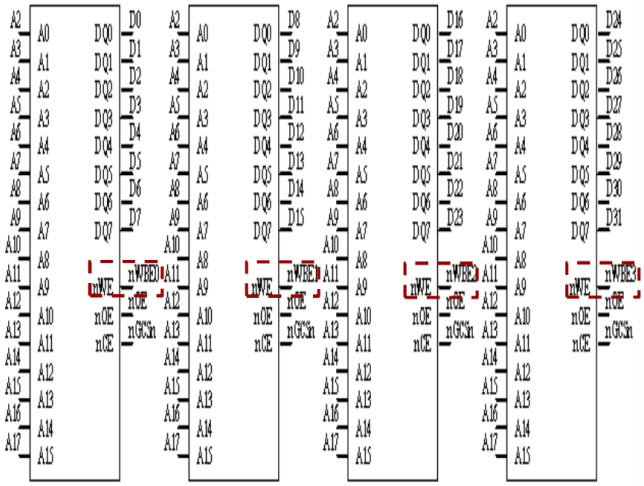
➢四片8位存储器芯片并联构成32位存储器系统（续）



- 四片存储器的地址线…A_1、A_0与S3C2440的地址总线…$ADDR_3$、$ADDR_2$相连。
- 四片存储器的允许输出信号nOE端都接S3C2440的nOE引脚。
- 四片存储器的片选信号nCE端都接S3C2440的nGCSn引脚。(作为一个整体配置到同一Bank中)

存储器芯片连接实例（续）

➢四片8位存储器芯片并联构成32位存储器系统（续）



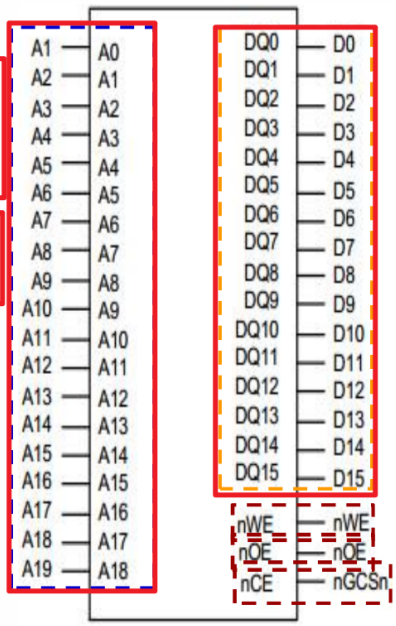
- 低8位存储器的写信号nWE端接S3C2440的$nWBE_0$引脚，用于写入低字节数据；
- 次低8位存储器的写信号nWE端接S3C2440的$nWBE_1$引脚，用于写入次低字节数据；
- 次高8位存储器的写信号nWE端接S3C2440的$nWBE_2$引脚，用于写入次高字节数据；
- 高8位存储器的写信号nWE端接S3C2440的$nWBE_3$引脚，用于写入高字节数据。

1片16位连接16位芯片：

存储器芯片连接实例（续）

➢16位存储器芯片构成16位存储器系统

- 存储器的16位数据线DQ_{15}～DQ_0与S3C2440的低16位数据总线$DATA_{15}$～$DATA_0$相连。
- 存储器的地址线…A_1、A_0与S3C2440的地址总线…$ADDR_2$、$ADDR_1$相连。
- 存储器的允许输出信号nOE端接S3C2440的nOE引脚。
- 存储器的写信号nWE端接S3C2440的nWE引脚。
- 存储器的片选信号nCE端接S3C2440的nGCSn引脚。



ROM存储器

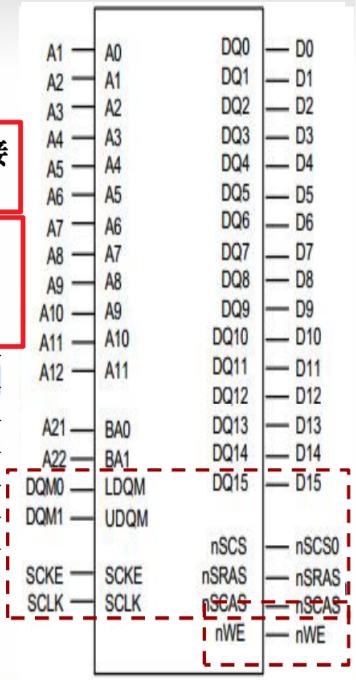
存储器芯片连接实例（续）

➢16位存储器芯片构成16位存储器系统

- SRAM存储器：还需存储器nUB、nLB端接S3C2440的nBE_1、nBE_0引脚。
- SDRAM存储器：还需保证存储器LDQM、UDQM、SCKE、SCLK、nSRAS、nSCAS、nSCS的正确连接。

只能映射到Bank6或Bank7。

nSRAS	O	SDRAM 行地址选通
nSCAS	O	SDRAM 列地址选通
nSCS[1:0]	O	SDRAM 片选
DQM[3:0]	O	SDRAM 数据屏蔽
SCLK[1:0]	O	SDRAM 时钟
SCKE	O	SDRAM 时钟使能



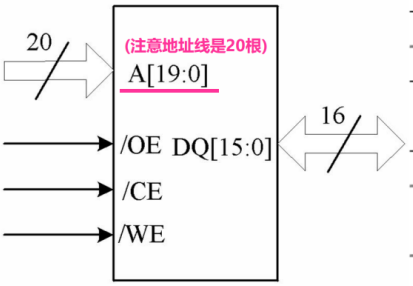
NOR Flash存储器芯片及其连接

NOR Flash特点：

- 读写快，具有芯片内执行XIP特性
- 写入慢，价格高
- 擦写次数10w次
- 带有SRAM接口，便于数据存取
- 适用于存储固化的**系统启动引导代码（Bootloader）**，操作系统代码，应用程序代码
- **通常配置到Bank0**，当系统上电或者复位后从其内部获得指令开始执行

NOR Flash存储器芯片——SST39VF1601

- 单片存储容量为2MB。以半字模式(16位数据宽度)方式工作。
- 工作电压2.7～3.6V。
- 采用48脚TSOP封装或48脚TFBG封装。



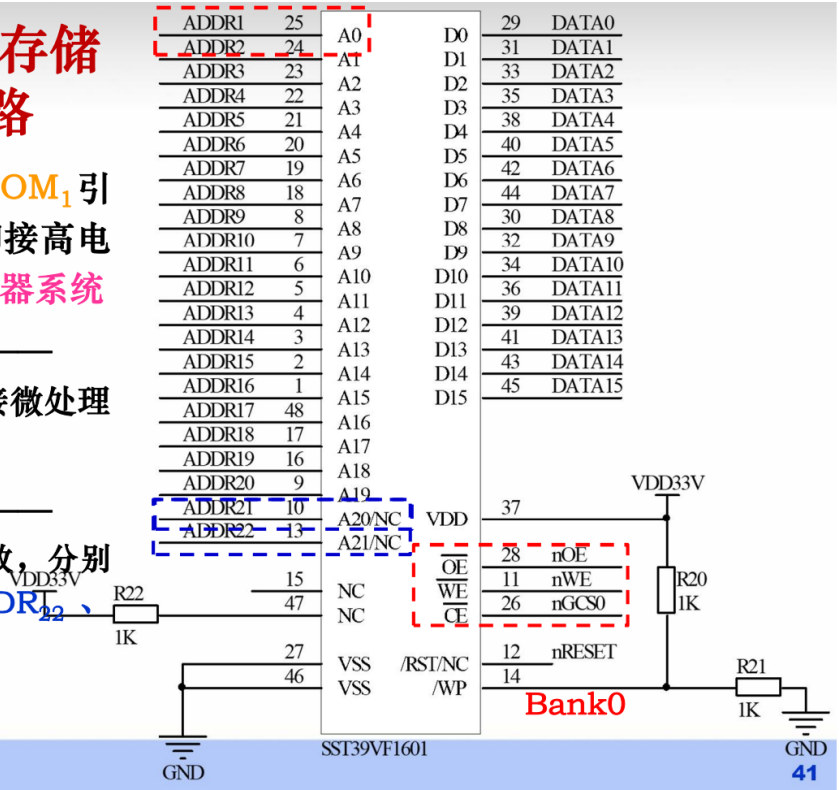
SST39VF1601逻辑框图

引脚	类型	描述
A[19 : 0]	I	20根地址总线A19~A0。
DQ[15 :0]	I/O（有三态功能）	数据总线，在读/写操作时提供16位的数据总线宽度
/CE	I	片选信号，低电平有效。当对SST39VF1601进行读/写操作时，该引脚必须为低电平；当为高电平时，芯片处于高阻状态。
/OE	I	允许输出使能，低电平有效。
/WE	I	写使能，低电平有效。当对SST39VF1601进行编程和擦除操作时，控制相应的写命令。
VDD	—	3.3V电源。
VSS	—	接地。

40

NOR Flash存储器的接口电路

- 微处理器芯片的OM_1引脚接地，OM_0引脚接高电平。——16位存储器系统
- SST39VF3201——4MB，A_{20}有效，接微处理器$ADDR_{21}$。
- SST39VF6401——8MB，A_{21}、A_{20}有效，分别接微处理器$ADDR_{22}$、$ADDR_{21}$。

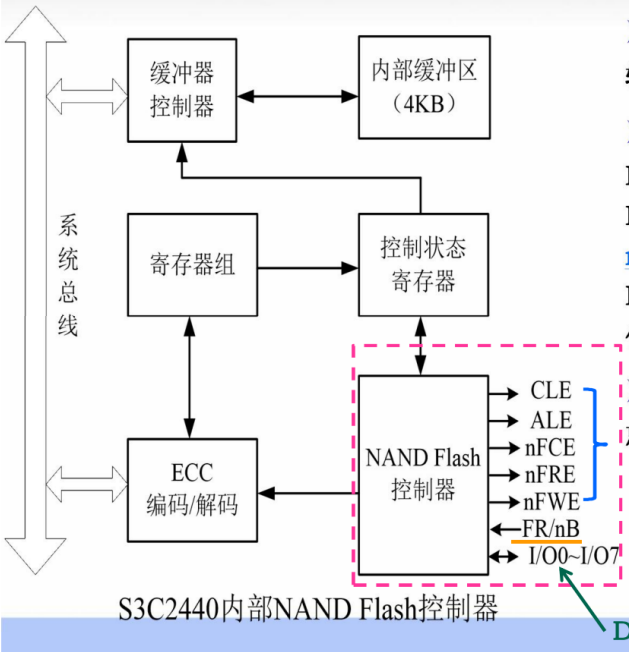


41

NAND Flash存储器芯片及其连接

- 擦除和写入速度快，价格便宜
- 使用时需要复杂 **I/O 接口电路** 和 存储器管理操作
  - **以页位最小单位进行读写，以块为最小单位进行擦除（若干页构成一个块）**
- 擦写100w次
- **支持自动启动引导**

8位NAND Flash控制器



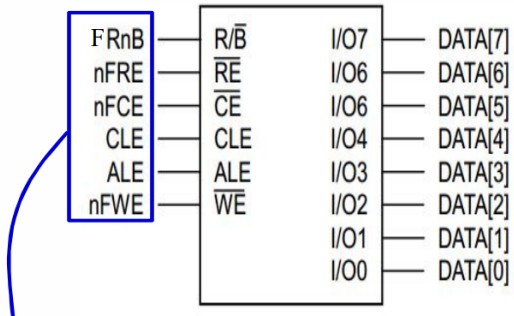
S3C2440内部NAND Flash控制器

- **I/O引脚**：I/O_0~I/O_7用于输出地址、输出命令、输入/输出数据。——分时复用
- **控制引脚**：CLE(Command Latch Enable)命令锁存使能；ALE(Address Latch Enable)地址锁存使能；nFCE(Chip Enable)片选；nFRE(Read Enable)读使能；nFWE(Write Enable)写使能。
- **状态引脚**：FR/nB(Ready/Busy)准备就绪或忙状态信号。
  - =0表示“忙”，程序不能对芯片进行操作；
  - =1表示“准备就绪”，程序可以操作芯片。

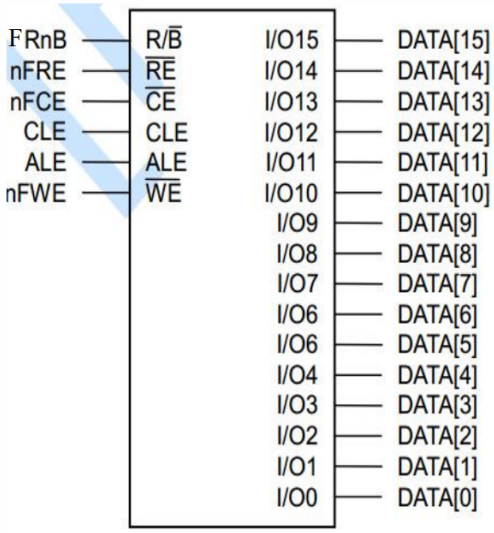
$DATA_0$~$DATA_7$

44

NAND Flash存储器芯片引脚(接口)



8位NAND Flash存储器

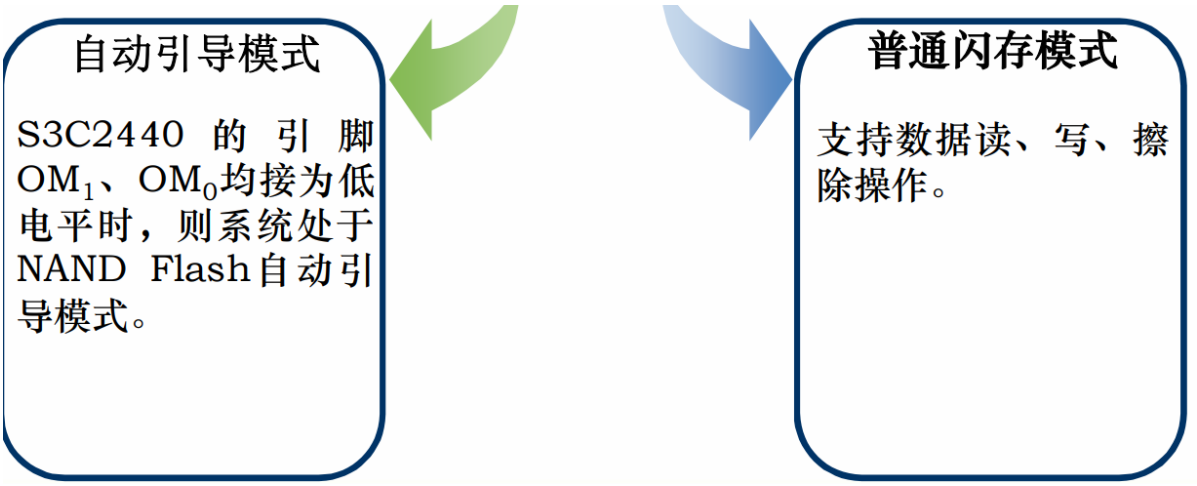


16位NAND Flash存储器

CLE	O	指令锁存使能
ALE	O	地址锁存使能
nFCE	O	Nand Flash 片选使能
nFRE	O	Nand Flash 读使能
nFWE	O	Nand Flash 写使能
FRnB	I	Nand Flash 就绪/忙

S3C2440微处理器芯片引脚定义

NAND Flash两种工作模式：


自动引导模式
S3C2440 的引脚 OM_1、OM_0均接为低电平时，则系统处于 NAND Flash自动引导模式。
普通闪存模式
支持数据读、写、擦除操作。

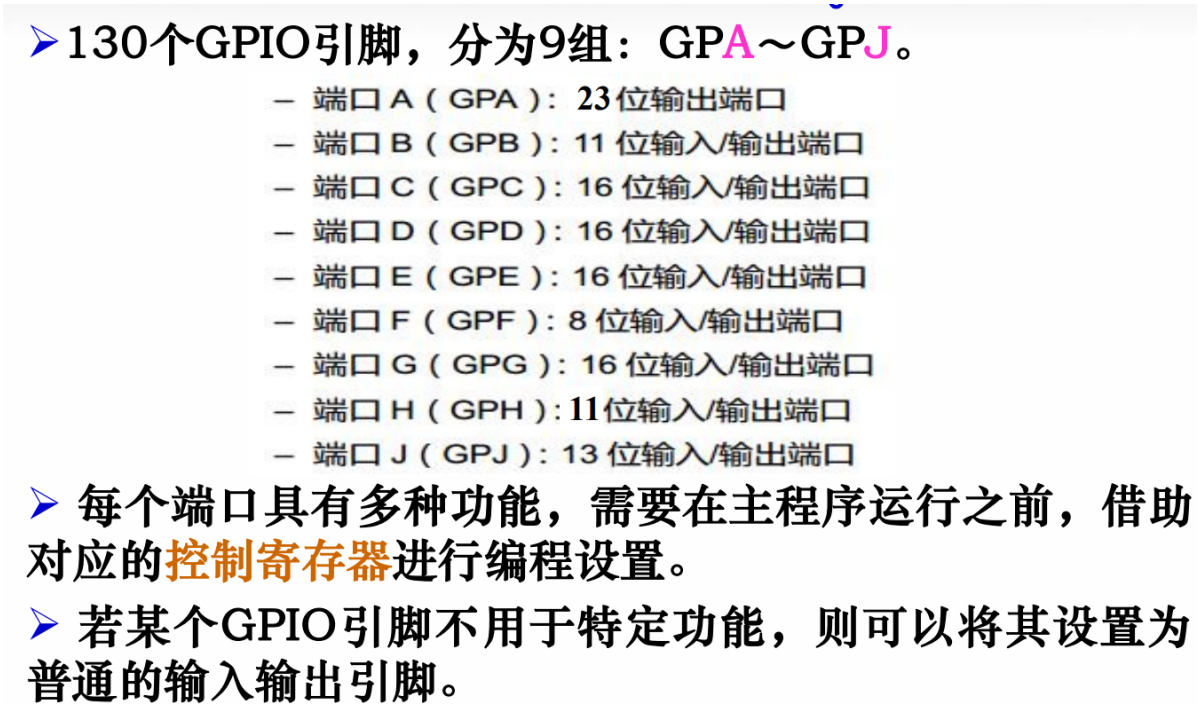
第五章：常用外围设备及接口

通用输入输出端口（GPIO）

通用输入输出端口（General Purpose I/O）是嵌入式系统重要组成部分。

用于连接各类输入，输出设备，实现其对微处理器之间的数据传输

构成：**130 个 GPIO 引脚,分为 9 组，分别是 GPA~GPJ** 每个端口都有与之对应的控制寄存器。



只掌握**GPF**和**GPG** 即可

端口G（GPG）

- 16个引脚。
- 三种功能：
  - ✓ 普通输入/输出口；
  - ✓ 用于中断请求信号。
  - ✓ 用于SPI总线、LCD显示器的控制信号。

使用GPIO点亮发光二极管：

3.汇编代码

```
.text
.global _start
_start:
        LDR R0,=0x56000050          ;端口F的配置寄存器地址
        LDR R1,[R0]
        BIC R1,R1,#0x3F00           ;配置GPF4～6为输出引脚
        ORR R1,R1,#0x1500
        STR R1,[R0]
        LDR R0,=0x56000054          ;端口F的数据寄存器地址
        LDR R1,[R0]
        BIC R1,R1,#0x70             ;GPF4～6输出为低电平
        STR R1,[R0]
WAIT:
        B WAIT
```

中断系统

概念

微处理器再执行正常程序的过程中，因某事件发生，收到来自外围部件的请求信号。若能够 响应该信号，则暂停当前程序的正常执行，转去执行针对请求事件的处理操作，待结束后再 返回被暂时中断的程序继续执行

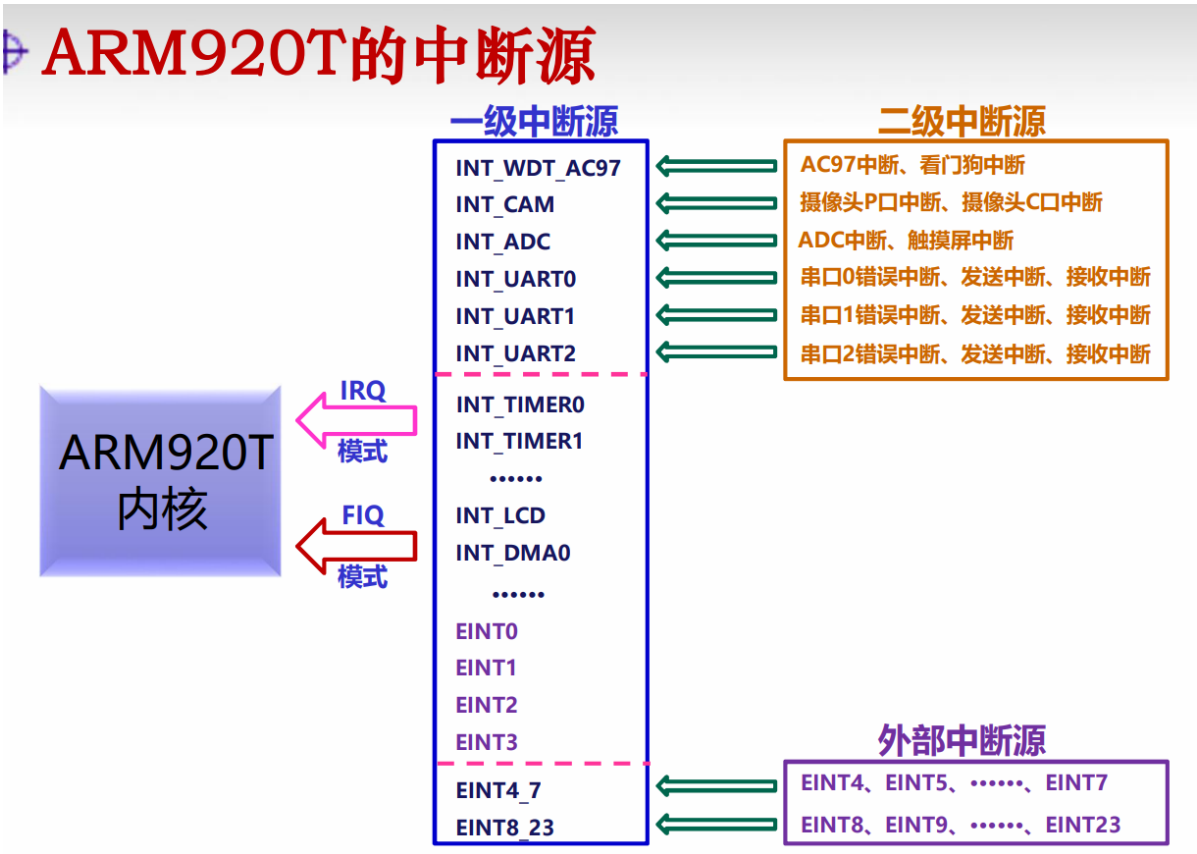
作用

1. **并行处理。**在外围设备需要传递数据时才产生中断，使得微处理器可以与多个外围设备同时工作，提高了微处理器的工作效率
2. **实时处理。**在实时控制系统中，外围设备提出服务请求的时间是随机的。只有通过中断系统，才能对他们快速响应。
3. **故障处理。**系统在运行中，常常会出现一些突发性故障，利用中断功能就可以对他们进行实时处理。

构成

ARM 的 7 种异常类型中，由芯片**外部中断请求引脚**和**内部外设**触发的是**外部中断请求（IRQ）** 和**快速中断请求（FIQ）**。

借助中断控制器处理 60 个中断源。



- 一级中断源：32个
- 二级中断源：15个

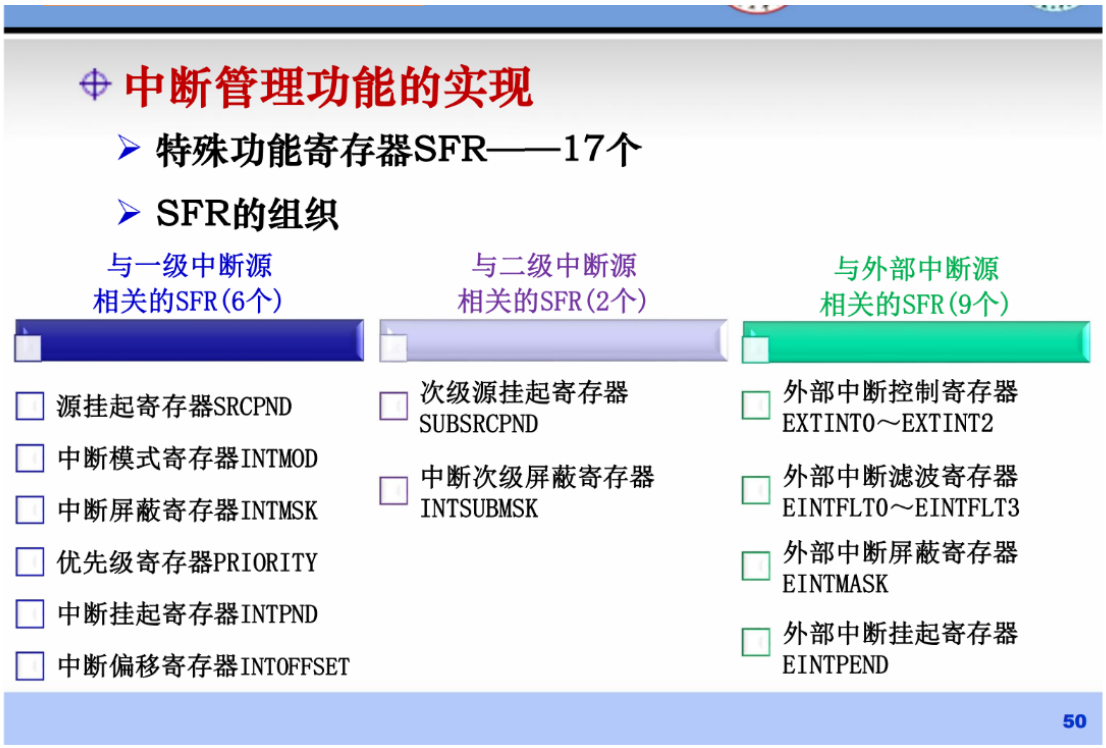
EINT8_23	外部中断8_23（16个）	ARB1
EINT4_7	外部中断4_7（4个）	ARB1
EINT3	外部中断3	ARB0
EINT2	外部中断2	ARB0
EINT1	外部中断1	ARB0
EINT0	外部中断0	ARB0

32个一级中断源包含了0-23个EINT外部中断源

中断控制器功能

- 外部中断请求信号处理
- 中断模式设定
- 中断请求信号标记
- 中断屏蔽设定
- 中断优先级管理
- 中断服务标记

17个特殊功能寄存器SFR（Special Function Register）



一级中断相关的SFR：

源挂起寄存器SRCPND：

- 用于标记**32个一级中断源的中断请求信号是否触发**，即是否有中断等待处理，供中断服务程序读取和判断。
- 需要人工清除置位，**写入1表示清除，0表示不变。**

中断模式寄存器INTMOD：

- 用于设定**32个一级中断源**的中断模式。
- 设置 1 按照FIQ模式。设置 0 按照IRQ模式。

中断屏蔽寄存器INMSK：

- 用于设定**32个一级中断源是否被允许中断**
- 设置 1 相应的中断源不会被服务。设置 0 相应的中断源可以被服务。

优先级寄存器PRIORITY：

- 用于设定**32个一级中断源在IRQ模式下的优先级顺序**
- 分别通过 **2位ARB_SEL（优先级顺序）**和 **1位ARB_MODE（优先级类型）** 来设定

中断优先级的类型

- **静态优先级。**——优先级固定
  - ✓ REQ1～REQ4的优先级一旦通过程序设定好，则固定不再变化。
- **动态优先级。**——优先级循环
  - ✓ REQ1～REQ4的优先级通过程序设定好后,只要被中断处理了，则其优先级自动降为最低。

中断挂起寄存器INTPND

- 用于**标记32个一级中断源**的中断请求是否即将或者正在被微处理器服务。
- = 1 表明所对应的中断在**所有已触发的中断**中优先级最高且没有被屏蔽
- 同一时刻只有1位被置1，**仅对IRQ模式的中断有效**
- 需要人工清除置位

中断偏移寄存器INTOFFSET

- 用于**标记 INTPND 中置“1”位对应中断源的偏移量，代表了即将或者正在**被微处理器服务的**中断源号。**
- 人工清除 SRCPND 和 INTPND 中的置 1 位，该寄存器的值会被自动清除。
- **仅对IRQ模式的中断有效**

二级中断相关的SFR：

与二级中断源相关的SFR

1.次级源挂起寄存器

寄存器	地址	R/W	描述	复位值
SUBSRCPND	0X4A000018	R/W	指示中断请求状态 0 = 未请求中断　　1 = 中断源已声明中断请求	0x00000000

2.中断次级屏蔽寄存器

寄存器	地址	R/W	描述	复位值
INTSUBMSK	0X4A00001C	R/W	决定屏蔽哪个中断源。被屏蔽的中断源将不会服务 0 = 中断服务可用　　1 = 屏蔽中断服务	0xFFFF

外部中断相关的SFR：

外部中断控制器EXTINT0~EXTINT2 共三个

➢ 外部中断控制寄存器——EXTINT0

名称	位索引	描述	初始值
EINTn (n=0~7)	[4n+2:4n]	设置EINTn的信号触发方式 000=低电平；001=高电平；01x=下降沿触发； 10x=上升沿触发；11x=双边沿触发	000

➢ 外部中断控制寄存器——EXTINT1

名称	位索引	描述	初始值
FLTENn (n=8~15)	[4n-29]	EINTn的滤波器使能 0=滤波器禁止；1=滤波器使能	0
EINTn (n=8~15)	[4n-30: 4n-32]	设置EINTn的信号触发方式 000=低电平；001=高电平；01x=下降沿触发； 10x=上升沿触发；11x=双边沿触发	000

➢ 外部中断控制寄存器——EXTINT2

名称	位索引	描述	初始值
FLTENn (n=16~23)	[4n-61]	EINTn的滤波器使能 0=滤波器禁止；1=滤波器使能	0
EINTn (n=16~23)	[4n-62: 4n-64]	设置EINTn的信号触发方式 000=低电平；001=高电平；01x=下降沿触发； 10x=上升沿触发；11x=双边沿触发	000

外部中断滤波寄存器EINTFLT0~EINTFLT3 共四个

外部中断滤波寄存器

- 用于设定8个外部中断请求信号（EINT16～EINT23）的滤波器的时钟信号来源和滤波宽度。
- 为了确保微处理器能有效检测到外部中断请求信号，要求信号的有效逻辑电平至少要保持40ns。

寄存器	地址	R/W	描述	复位值
EINTFLT0	0x56000094	R/W	保留	0x000000
EINTFLT1	0x56000098	R/W	保留	0x000000
EINTFLT2	0x5600009C	R/W	外部中断滤波寄存器 2	0x000000
EINTFLT3	0x560000A0	R/W	外部中断滤波寄存器 3	0x000000

外部中断滤波寄存器——EINTFLT2

名称	位索引	描述	初始值
FLTCLKn (n=16~19)	[8n-121]	设置EINTn的滤波器的时钟信号来源 0=PCLK；1=EXTCLK/OSC_CLK。	0
EINTFLTn (n=16~19)	[8n-122: 8n-128]	设置EINTn的滤波宽度	0x00

外部中断滤波寄存器——EINTFLT3

名称	位索引	描述	初始值
FLTCLKn (n=20~23)	[8n-153]	设置EINTn的滤波器的时钟信号来源 0=PCLK；1=EXTCLK/OSC_CLK。	0
EINTFLTn (n=20~23)	[8n-154: 8n-160]	设置EINTn的滤波宽度	0x00

外部中断屏蔽寄存器(EINTMASK)

➢ 用于设定20个外部中断请求信号（EINT4～EINT23）是否允许中断。

寄存器	地址	R/W	描述	复位值
EINTMASK	0x560000A4	R/W	外部中断屏蔽寄存器	0x000FFFFF

➢ 外部中断屏蔽寄存器——EINTMASK

名称	位索引	描述	初始值
EINTn (n=20~23)	[n]	设置EINTn的中断允许 0=使能中断；1=禁止中断。	0
EINTn (n=4~19)	[n]	设置EINTn的中断允许 0=使能中断；1=禁止中断。	1
保留	[3:0]	保留	111

外部中断挂起寄存器（EINTPEND）：

➢ 用于标记20个外部中断请求信号（EINT4～EINT23）是否触发，供中断服务程序读取和判断。

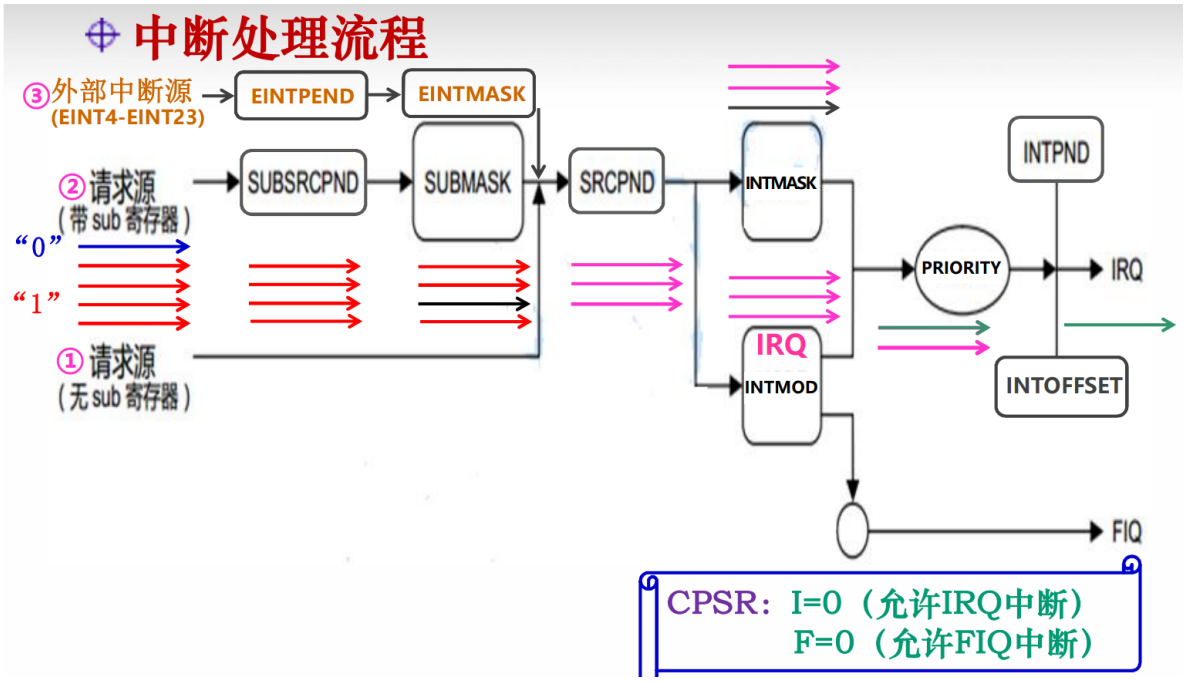
寄存器	地址	R/W	描述	复位值
EINTPEND	0x560000A8	R/W	外部中断挂起寄存器	0x00

➢ 外部中断挂起寄存器——EINTPEND

名称	位索引	描述	初始值
EINTn (n=4~23)	[n]	表明EINTn引脚中断请求信号是否有效 0=无中断请求；1=有中断请求。	0
保留	[3:0]	保留	000

△注意：对某位写入“1”，可实现对其清零。

中断处理流程



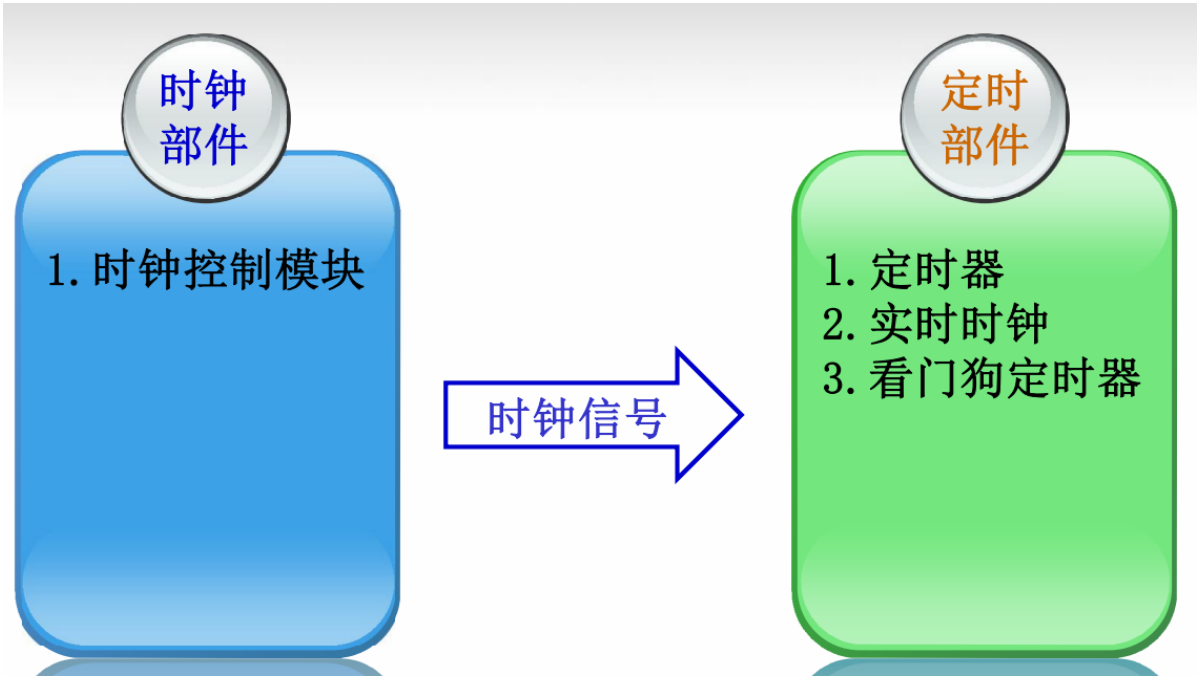
定时部件

时钟部件

- 为处理器工作提供基本的时钟信号，以实现其内部功能电路及外围设备的时序控制

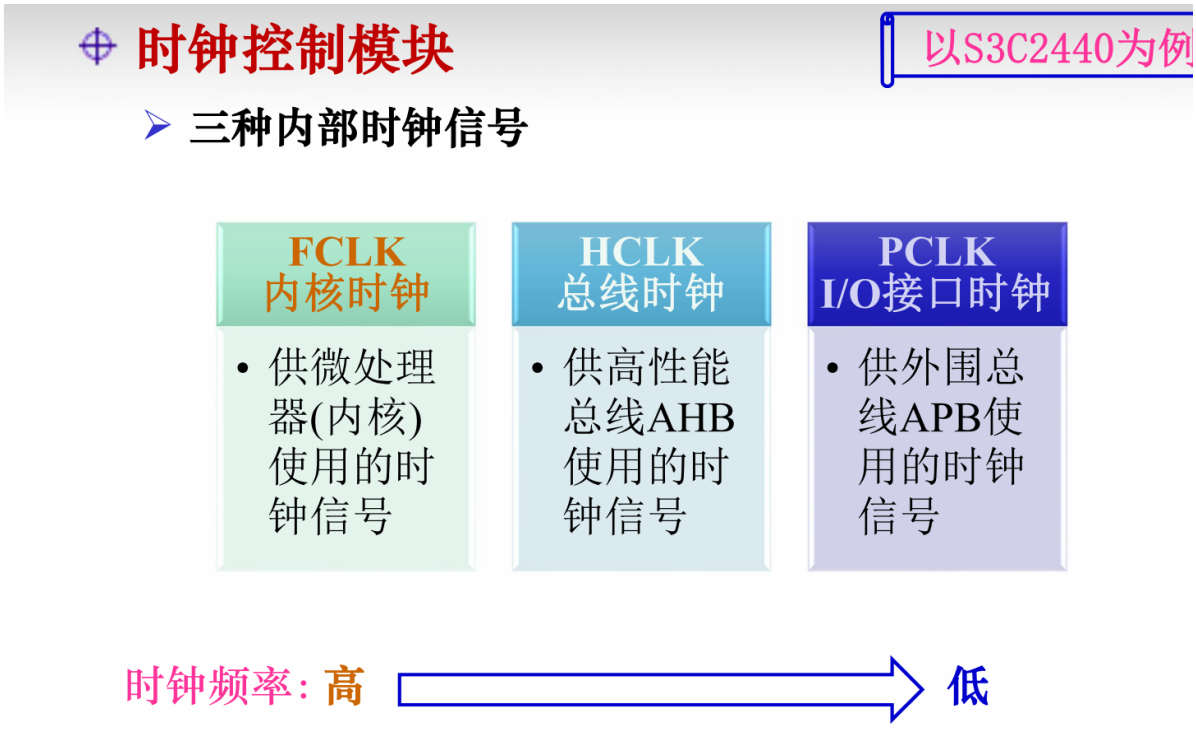
定时部件

- 产生不同周期或特定波形的时钟信号，满足不同的实际应用需求。
- 对外部输入信号进行计数
- 为系统提供时间信息，Y,M,D,M,S 。。。。
- 当系统出现故障，为各控制器提供复位信号

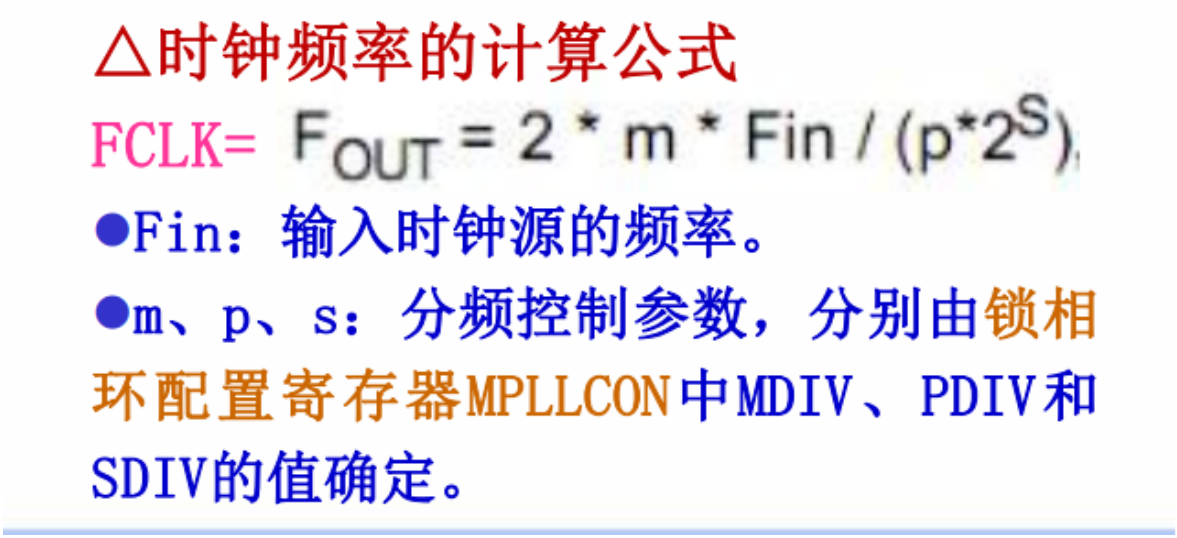


时钟部件通过时钟信号控制定时部件。

三种内部时钟信号：



时钟频率计算公式：



设定三种时钟信号的比例关系

时钟控制模块(续)

➢ 分频比例的选择——设定三种时钟信号的比例关系

时钟分频控制（CLKDIVN）寄存器

寄存器	地址	R/W	描述	复位值
CLKDIVN	0x4C000014	R/W	时钟分频控制寄存器	0x00000004

CLKDIVN	位	描述	初始状态
DIVN_UPLL	[3]	UCLK 选择寄存器（UCLK 必须为 48MHz 给 USB） 0：UCLK = UPLL 时钟　　1：UCLK = UPLL 时钟 / 2 当 UPLL 时钟被设置为 48MHz 时，设置为 0 当 UPLL 时钟被设置为 96MHz 时，设置为 1	0
HDIVN	[2:1]	00：HCLK = FCLK/1 01：HCLK = FCLK/2 10：HCLK = FCLK/4　当 CAMDIVN[9] = 0 时 HCLK = FCLK/8　当 CAMDIVN[9] = 1 时 11：HCLK = FCLK/3　当 CAMDIVN[8] = 0 时 HCLK = FCLK/6　当 CAMDIVN[8] = 1 时	00
PDIVN	[0]	0：PCLK 是和 HCLK/1 相同的时钟 1：PCLK 是和 HCLK/2 相同的时钟	0

经典例题:

➢ 举例：时钟信号频率计算

△已知时钟源的频率为12MHz，MPLLCON中MDIV=92，PDIV=1，SDIV=1，且CLKDIVN中PCLK:HCLK:FCLK设置为1:2:8。试计算FCLK、HCLK和PCLK的频率。

分频参数：m = (MDIV + 8), p = (PDIV + 2), s = SDIV

=92+8　　=1+2　　=1

FCLK＝2*(92+8)*(12000000)/(3×2^1)=400000000=400MHz

HCLK=400/4=100MHz

PCLK=400/8=50MHz

时钟控制模块(续)

➢ 举例：相关寄存器的设定程序

```
;设置锁相环配置寄存器MPLLCON
ldr   r0,  = MPLLCON
ldr   r1,  = ((92 << 12) + (1 << 4) + 1)
str   r1,  [r0]

;设置时钟分频控制寄存器CLKDIVN
ldr   r0,=CLKDIVN
ldr   r1,=0x5          ;0x5=0101b
str   r1,[r0]
```

MPLLCON	Bit	
MDIV	[19:12]	Main divider control
PDIV	[9:4]	Pre-divider control
SDIV	[1:0]	Post divider control

CLKDIVN	Bit	Description
DIVN_UPLL	[3]	UCLK select register(UCLK must be 48MHz for USB) 0: UCLK = UPLL clock 1: UCLK = UPLL clock / 2 Set to 0, when UPLL clock is set as 48MHz Set to 1. when UPLL clock is set as 96MHz.
HDIVN	[2:1]	00 : HCLK = FCLK/1. 01 : HCLK = FCLK/2. 10 : HCLK = FCLK/4 when CAMDIVN[9] = 0. HCLK= FCLK/8 when CAMDIVN[9] = 1. 11 : HCLK = FCLK/3 when CAMDIVN[8] = 0. HCLK = FCLK/6 when CAMDIVN[8] = 1.
PDIVN	[0]	0: PCLK has the clock same as the HCLK/1. 1: PCLK has the clock same as the HCLK/2.

定时器

- 功能：定时，计数，脉宽调制
- 构成：5个16位定时器**（递减计数器）**
- 每个定时器都有一个可以生成 **5** 种不同分频信号（**1/2、1/4、1/8、1/16 和外部时钟 TCLK**）的时钟分频器
- 定时器0和 1 共用一个 8 位预分频器，2、3、4 共用另外一个。

工作原理：

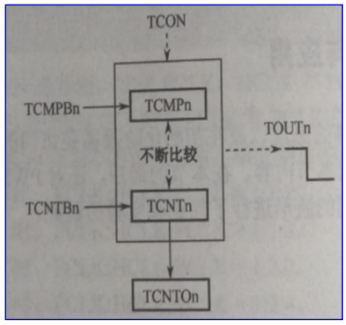
定时器的工作原理

➢ 实际执行减1操作的是定时计数寄存器TCNTn。为了防止对其读写影响计数，不允许程序对其直接访问。

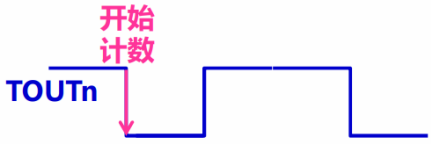
✓ 将计数初值写入定时计数缓冲寄存器TCNTBn。每次定时开始时，其值会被复制到TCNTn中。（同时TCMPBn→TCMPn）

✓ 读取TCNTBn，读出的是下次定时操作的重载值，并非TCNTn的当前计数值。

✓ 读取定时计数监视寄存器TCNTOn，可以间接获取TCNTn的动态计数值。

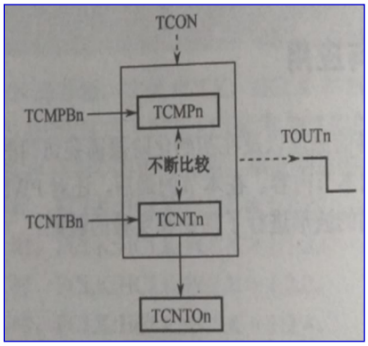


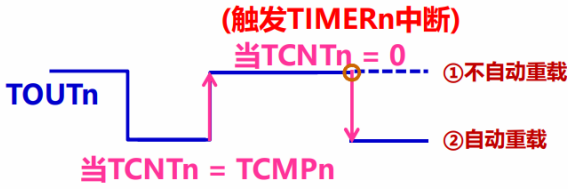
➢ 启动计数后，TOUTn引脚的输出信号进行第一次反相。



定时器的工作原理(续)

- 当TCNTn等于定时比较寄存器TCMPn时，TOUTn引脚的输出信号进行第二次反相。
- TCNTn减到0时，若没有屏蔽允许，则触发TIMERn中断。并根据定时器控制寄存器TCON的“自动重载”标志，决定是否复制TCNTBn到TCNTn中，以开启下一次定时操作。

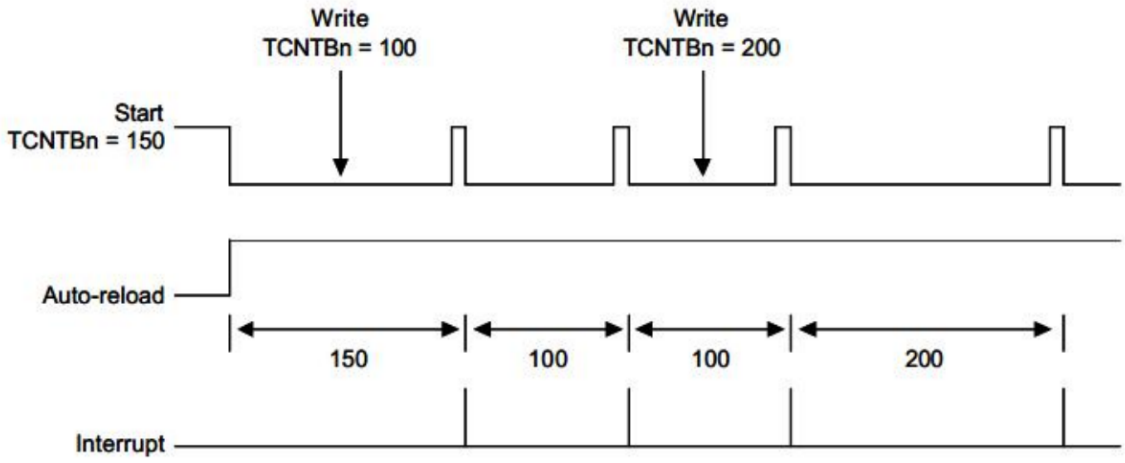




定时器的工作原理(续)

- TCNTBn和TCMPBn有双缓冲功能，故在当前定时操作中设置的新的定时器值，仅当当前定时操作执行完毕后才有效。

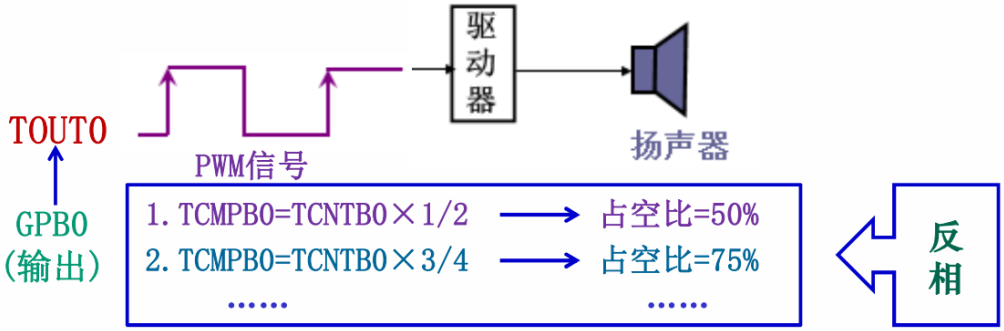
——非立即有效



需要掌握一个案例：编程实现使用定时器实现不同占空比下蜂鸣器的发声。

定时器应用实例

- 利用定时器0的PWM功能实现扬声器(蜂鸣器)发声。



//1.配置GPB0为定时器0输出，并保持端口B的其它引脚输入输出不变。

GPB0	[1:0]	00 = 输入	01 = 输出	10 = TOUT0	11 = 保留

```
LDR R0,=0x56000010      ;端口B的配置寄存器GPBCON地址
LDR R1,[R0]
BIC R1,R1,#0x03         ;将寄存器的[1:0]=00
ORR R1,R1,#0x02         ;将寄存器的[1:0]=10
STR R1,[R0]
```

定时器应用实例(续)

//2.配置TCFG0和TCFG1，写入预分频和分频系数，以获取所需频率的定时器0输出信号TOUT0。

Prescaler 0	[7:0]	该 8 位决定了定时器 0 和 1 的预分频值

```
LDR R0,=0x51000000      ;定时器配置寄存器TCFG0地址
LDR R1,[R0]
BIC R1,R1,#0xFF         ;将寄存器的[7:0]=0x00
ORR R1,R1,#0x63         ;将寄存器的[7:0]=0x63(十进制99)
STR R1,[R0]
```

MUX 0	[3:0]	选择 PWM 定时器 0 的选通输入 0000 = 1/2　0001 = 1/4　0010 = 1/8　0011 = 1/16 01xx = 外部 TCLK1

```
LDR R0,=0x51000004      ;定时器配置寄存器TCFG1地址
LDR R1,[R0]
BIC R1,R1,#0x0F         ;将寄存器的[3:0]=0x0
ORR R1,R1,#0x02         ;将寄存器的[3:0]=0x2 （二进制0010）
STR R1,[R0]
```

定时器应用实例(续)

//3.配置TCNTB0和TCMPB0，写入计数初值和决定脉宽的比较值。

定时器 0 计数缓冲寄存器	[15:0]	设置定时器 0 的计数缓冲器的值

```
LDR R0,=0x5100000C      ;定时器0计数缓冲寄存器TCNTB0地址
MOV R1,#62500           ;将计算得到的计数初值写入TCNTB0
STR R1,[R0]
```

定时器 0 比较缓冲寄存器	[15:0]	设置定时器 0 的比较缓冲器的值

```
LDR R0,=0x51000010      ;定时器0比较缓冲寄存器TCMPB0地址
MOV R1,R1,LSR #1        ;将TCNTB0/2写入TCMPB0，设置PWM信号占空比50%
STR R1,[R0]
```

定时器应用实例(续)

//4.配置TCON，设置自动重载，并启动定时器0。

定时器 0 自动重载开/关	[3]	决定定时器 0 的自动重载开启或关闭 0 = 单稳态　1 = 间隙模式（自动重载）
定时器 0 输出变相开/关	[2]	决定定时器 0 的输出变相开启或关闭 0 = 关闭变相　1 = TOUT0 变换极性
定时器 0 手动更新（注释）	[1]	决定定时器 0 的手动更新 0 = 无操作　1 = 更新 TCNTB0 和 TCMPB0
定时器 0 启动/停止	[0]	决定定时器 0 的启动或停止 0 = 停止定时器 0　1 = 启动定时器 0

```
LDR R0,=0x51000008      ;定时器控制寄存器TCON地址
LDR R1,[R0]
ORR R1,R1,#0x02         ;TCON[1]=1：通过设置手动更新，实现首次计数值
                        ;的加载(从TCNTBn→TCNTn，从TCMPBn→TCMPn)。
STR R1,[R0]
BIC R1,R1,#0x02         ;TCON[1]=0：清除手动更新，下一次才能写入。
STR R1,[R0]
ORR R1,R1,#0x0D         ;TCON[3]=1：自动重载；TCON[2]=1：反相；
                        ;TCON[0]=1：启动定时器0。
STR R1,[R0]
```

TCON[2]：输出是否变相，取决于实际硬件电路需要

定时器应用实例(续)

//5.配置中断模式寄存器INTMOD，设置普通中断；配置中断屏蔽寄存器INTMSK，允许定时器0中断。

INT_TIMER0	[10]	0 = IRQ	1 = FIQ

INT_TIMER0	[10]	0 = 可服务	1 = 屏蔽

//6.在定时器0计数值达到0并触发中断后，在IRQ异常处理程序（中断服务程序）中读取中断挂起寄存器INTPND，判断是否是定时器0的中断。若是，根据需要修改计数初值或比较值，并写入TCNTBn和TCMPBn。

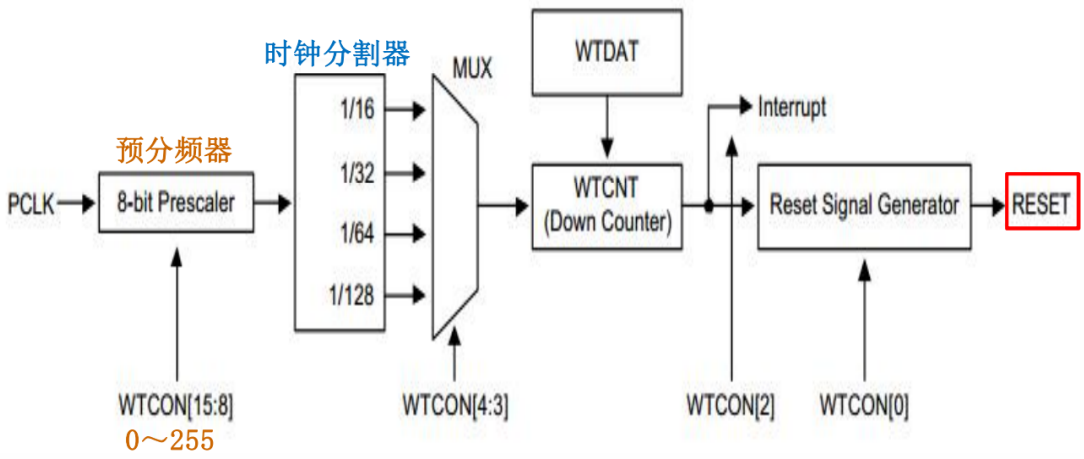
INT_TIMER0	[10]	0 = 未请求	1 = 请求

看门狗定时器WDT

- 一种用于当噪声或系统错误引起故障时恢复控制器操作的定时器。
- 本质上是一个16**位内部定时器，计数器值为 0 时**通过**触发中断服务**激活 **128 个 PCLK 时钟周期的内部复位信号。**
- 可用作一个普通的带中断请求的16位定时器
- 只能用**PCLK时钟信号**作为源输入时钟信号，且没有对外的输出信号引脚
- 系统处于**嵌入式ICE调试模式**时，WDT必须误操作

内部结构：

看门狗定时器的内部结构



F_watchdog = PCLK / (Prescaler value + 1) / Division_factor

WTDAT 相当于 TCNTBn，WTCNT 相当于 TCNTn，但是 WTCNT 必须要被设定一个初值

看门狗定时器控制寄存器——WTCON

名称	位索引	描述	初始值
预分频	[15:8]	预分频值(0到255)	0x80
保留	[7:6]	要求工作时这两位必须为00	00
看门狗定时器	[5]	看门狗定时器的使能或禁止位 0 = 禁止；1 = 使能	1
时钟选择	[4:3]	设定时钟分割系数 00 = 1/16；01 = 1/32；10 = 1/64；11 = 1/128	00
中断使能/禁止	[2]	中断的使能或禁止位 0 = 禁止；1 = 使能	0
保留	[1]	要求工作时此位必须为0	0
复位使能/禁止	[0]	看门狗定时器复位输出的使能或禁止位 0 = 禁止看门狗定时器的复位功能； 1 = 看门狗超时时发出 S3C2440A 复位信号	1

➢ 看门狗定时器数据寄存器——WTDAT

名称	位索引	描述	初始值
计数重载值	[15:0]	看门狗定时器重载的计数值(0～65535)	0x8000

△计数值=所需时间间隔÷T_watchdog = 所需时间间隔×F_watchdog

➢ 看门狗定时器计数寄存器——WTCNT

名称	位索引	描述	初始值
计数值	[15:0]	看门狗定时器的当前计数值	0x8000

注意事项：

看门狗定时器的注意事项

➢看门狗定时器中断与看门狗定时器复位

✓看门狗定时器中断是看门狗定时器作为普通定时器应用时，向微处理器发出内部中断请求；

✓看门狗定时器复位是让微处理器复位，相当于重新启动所有程序。

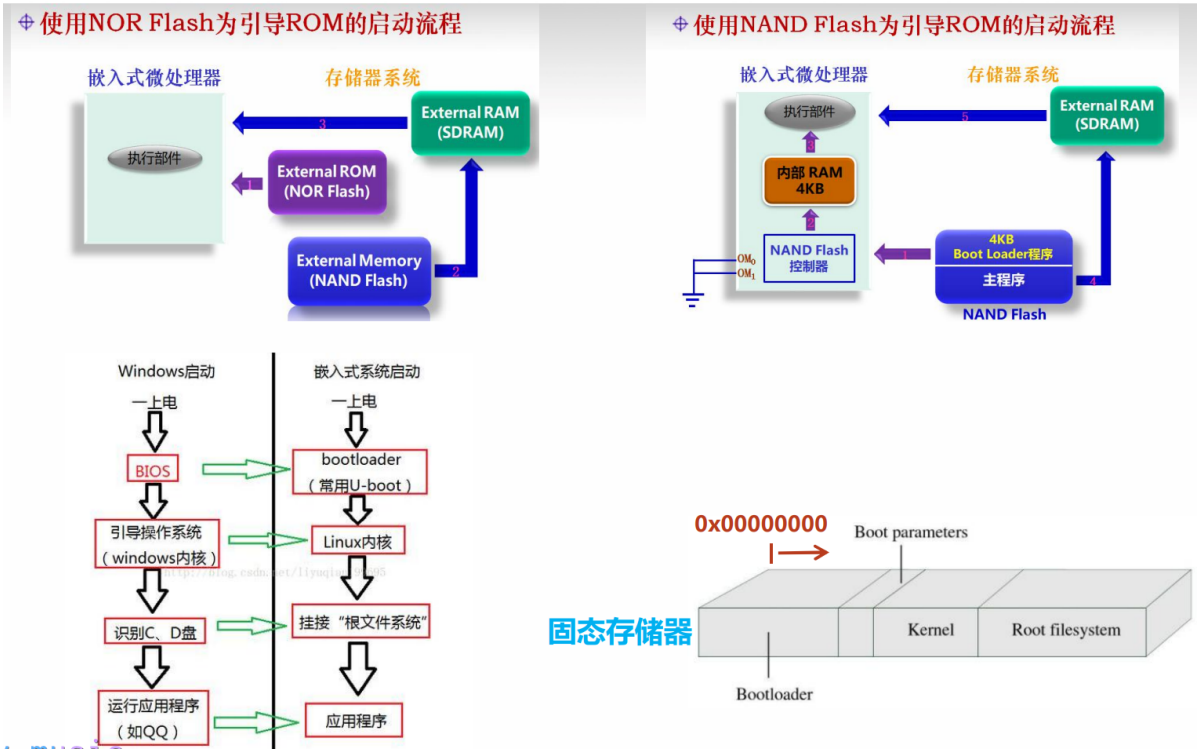
➢WTDAT存放WDT的计数初值，相当于普通定时器的TCNTBn。

➢WTCNT是WDT的减1计数器，相当于普通定时器的TCNTn(程序不可直接访问)。

➢不同于普通定时器，WTDAT的值在WDT初始使能时，不能自动装载到WTCNT中，所以必须要给WTCNT设定一个初始值。

第六章：嵌入式Linux系统及其编程

嵌入式系统启动流程



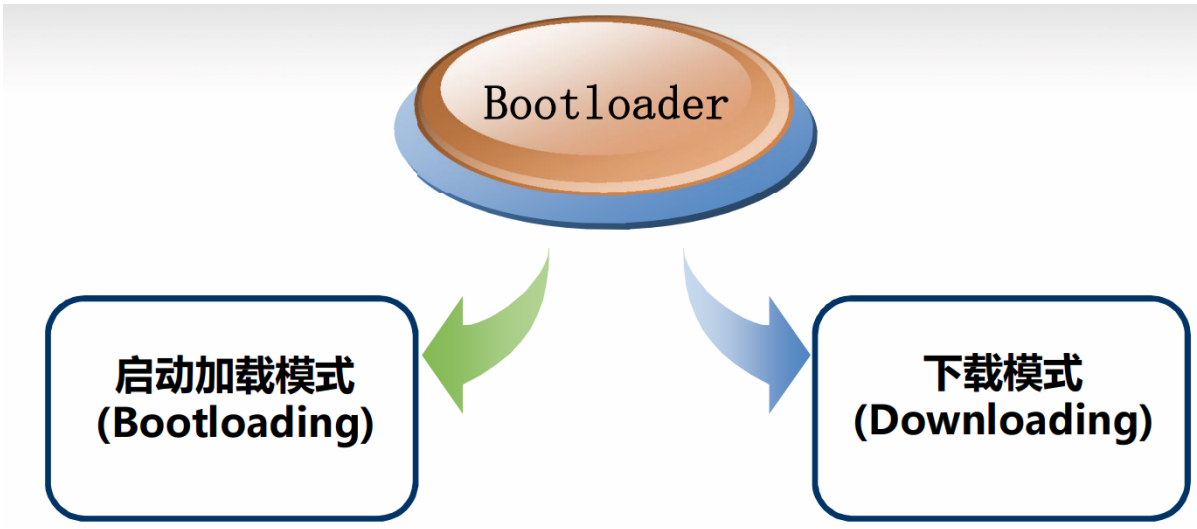
Bootloader概念

Bootloader(引导装载系统)时嵌入式系统上电以后，**操作系统内核**运行之前运行的第一段程序，类似Window的BIOS

- 是为了对系统的的硬件设备进行初始化，将操作系统映像转载到内存中，建立内存空间映射图，**为最终调用操作系统内核准备好正确的软硬件环境**

应用程序
根文件系统
中间件
内核
驱动
Bootloader
硬件

Bootloader两种运行模式



启动加载模式和**下载模式**。

启动加载模式：

- 也成为自主模式，是Bootloader的正常工作模式

流程：

1. 准备好操作系统内核运行所需要的环境和参数
2. 从目标机自身某个固态存储设备上将操作系统内核加载到RAM中
3. 在RAM中运行操作系统内核

下载模式：

- 由于系统需要修改/升级。**目标机**上的Bootloader通过窗口或者网络等通信手段，从**主机（通用计算机）**上下载文件到目标机的RAM中，然后写入目标机的Flash类固态存储设备中 -- **刷机**

■ 通信设备与协议

- 串口：xmodem / ymodem / zmodem协议中的一种。
- 以太网：TFTP协议。

■ 此模式下通常会向终端用户提供一个简单的命令行接口。

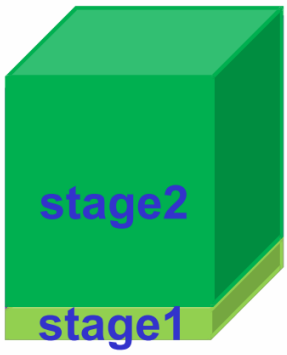
Bootloader组成，工作

Bootloader通常分为**stage1**和**stage2**两部分

stage1——依赖于CPU体系结构的代码，例如设备初始化代码等。通常用汇编语言来实现，以达到短小精悍的目的。

stage2——通常用C语言来实现，可以实现更复杂的功能，而且代码会具有更好的可读性和可移植性。

分级载入机制



stage1 和 stage2 的大体执行流程：

stage1通常包括以下步骤(按执行的先后顺序)：

- 硬件设备初始化。
- 为加载Bootloader的stage2准备RAM空间。
- 拷贝Bootloader的stage2到RAM空间中。
- 设置堆栈。
- 跳转到stage2的C程序入口点。

stage2通常包括以下步骤(按执行的先后顺序)：

- 初始化本阶段要使用到的硬件设备。
- 检测系统内存映射(memory map)。
- 将内核(kernel)映像和根文件系统映像从Flash存储器读到RAM空间中。
- 为内核设置启动参数。
- 调用内核。

stage1的具体过程

1. 基本的硬件初始化：为stage2的执行以及随后内核的执行准备基本的硬件环境。

(1) 屏蔽所有的中断

- 在 Bootloader执行的全过程中，可以不必响应任何中断；
- 中断屏蔽可通过设置微处理器内部**状态寄存器**的中断允许位来实现。

(2) 设置微处理器的速度和时钟频率

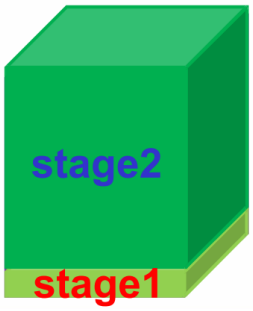
(3) RAM初始化

- 设置内存控制器的相关功能寄存器及控制寄存器等。

(4) 初始化LED

- 表明系统的状态（OK还是Error）。

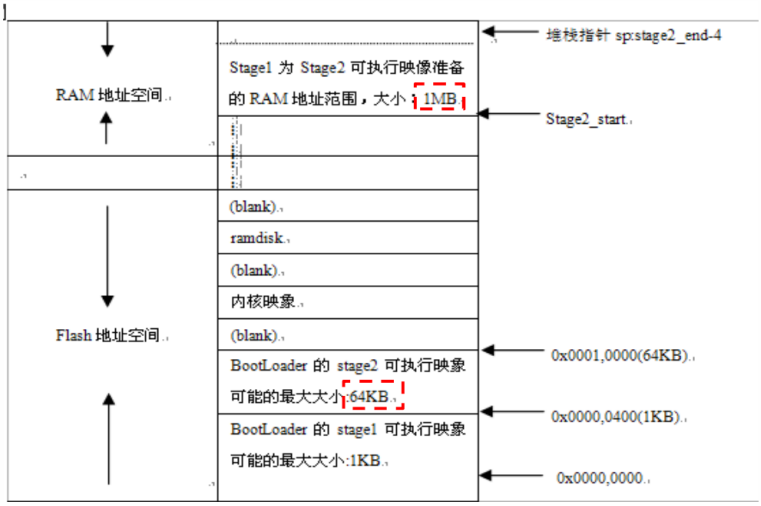
(5) 关闭微处理器内部指令/数据Cache



stage1的具体过程(续)

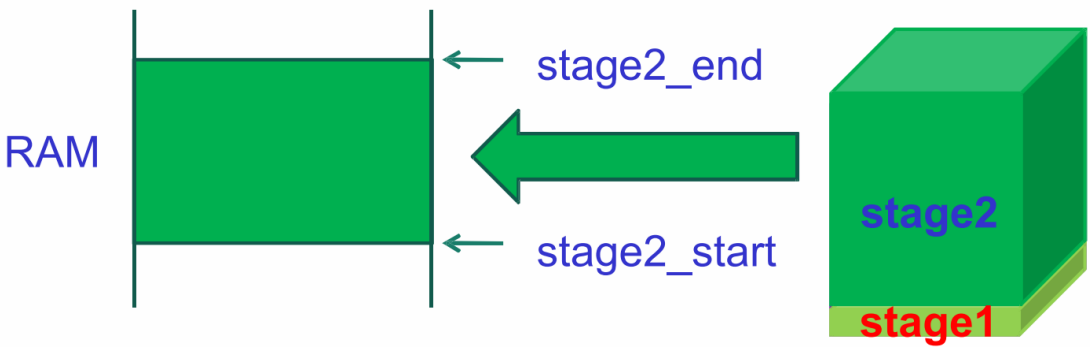
2. 为加载stage2 准备RAM空间

- 为了加快stage2的执行速度，通常需要把stage2加载到RAM空间中执行，因此必须为其准备好一段可用的RAM空间范围。



stage1的具体过程(续)

3. 拷贝stage2到RAM中



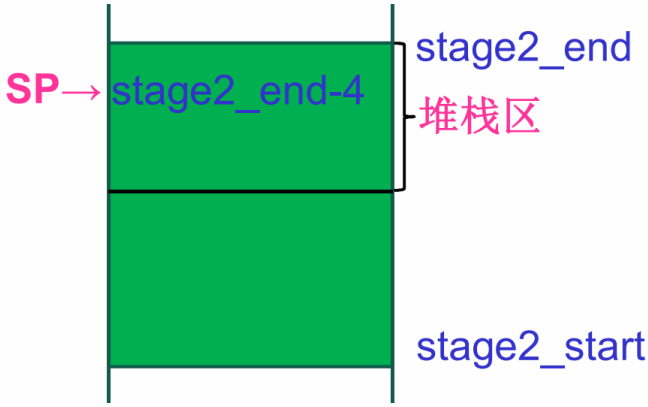
stage1的具体过程(续)

4. 设置堆栈指针SP，为执行C语言代码作好准备。

- SP指向stage2的RAM空间的最顶端；
- 向下生长型堆栈：SP指向stage2_end-4；
- ARM支持全部4种堆栈方式——满递增、空递增、满递减、空递减。

5. 跳转到stage2的C程序入口点。

- 对于ARM系统，可以通过修改PC寄存器为合适的地址来实现。



stage2具体执行流程

stage2的具体过程

1. 初始化本阶段要使用到的硬件设备

- 初始化至少一个串口，以便和终端用户进行I/O输出信息；
- 初始化定时器等外围部件(设备)；
- 设备初始化完成后，可以输出一些打印信息，例如：程序名称字符串、版本号等。

2. 检测系统的内存映射(Memory map)

- 在整个4GB物理地址空间中，有哪些地址范围被分配用来寻址系统的RAM单元。
- 不同微处理器有不同的内存映射，必须准确识别。

stage2的具体过程(续)

3. 加载内核映像(image文件)和根文件系统映像(image文件)

- 规划RAM的存储空间布局。
  包括内核映像所占用的RAM范围；根文件系统映像所占用的RAM范围。
- 从Flash存储器上拷贝内核映像和根文件系统映像到RAM中。

4. 设置内核的启动参数

- 将内核映像和根文件系统映像拷贝到RAM空间中后，可以准备运行系统内核。但在调用内核之前，需要设置内核的启动参数。
- 方法：把启动参数封装在预定好的**数据结构**里，拷贝到RAM的某个地址，并通过通用寄存器R2向内核传递这个地址值。

stage2的具体过程(续)

5. 调用内核

- 直接跳转到内核的第一条指令处。
- 应满足跳转条件：

(1) **微处理器的寄存器的设置：**
R0 = 0；
R1 = 机器类型ID；
R2 = 启动参数标记列表在RAM中起始基地址。

(2) **微处理器模式的设置：**
必须禁止中断（IRQ和FIQ）； ← **CPSR寄存器的控制位**
必须处于SVC模式。

(3) **Cache和MMU的设置：**
MMU必须关闭*； ← **CP15的C1寄存器的[0]位=0**
指令Cache可以打开也可以关闭；
数据Cache必须关闭。

Linux

五个主要子系统：

- **进程调度**：负责控制进程访问 CPU，Linux 采用基于优先级的进程调度方法。程序是一组指令的有序集合，是静态的，而进程是具有独立功能的程序的一次运行过程，是动态的。Linux提供了抢占式的多任务模式，由调度程序决定什么时候停止一个进程的运行，以便其他进程能够得到执行机会，这个强制的挂起动作称为抢占。
- **进程间通信**：在不同进程之间传播或交换信息，通过在内核中开辟一块缓冲区，不同进程通过缓冲区实现数据交换。
- **内存管理**：使多个进程安全地共享内存。
- **虚拟文件系统**：为文件系统提供了一个通用的接口，提供了内核中的一个抽象功能。
- **网络接口**，在设计上遵循模拟协议本身的分层体系结构。

Linux文件类型：

- **普通文件**（又分文本文件、二进制文件）
- **目录文件**（存储一组相关文件位置、大小等与文件有关的信息）
- **设备文件** （Linux把每一个I/O设备都看成一个文件）

Linux命令：

有些简单的需要掌握：

命令	作用
`ls`	列出目录内容，常用参数 `-l`（详细），`-a`（包含隐藏文件）
`cd`	切换目录，如 `cd /etc`，`cd ..`（上一级）
`pwd`	显示当前路径

命令	作用
`mkdir`	创建目录，如 `mkdir test`
`rmdir`	删除空目录
`rm`	删除文件或目录，`rm -r` 递归删除目录，`rm -f` 强制删除
`cp`	拷贝文件或目录，`cp -r` 拷贝目录
`mv`	移动或重命名文件
`su`	切换用户
`useradd`	添加用户
`tar`	打包与压缩
`chmod`	改变权限

命令	作用
`tar`	用于打包和压缩/解压文件，常用于 `.tar`、`.tar.gz` 文件的处理
`chmod`	修改文件或目录的权限（如可读、可写、可执行）
`echo`	输出文本或变量值，常用于脚本打印、重定向到文件
`man`	查看 Linux 命令的使用手册，是最权威的命令帮助系统